AI VIET NAM
@aivietnam.edu.vn

Linear Regression

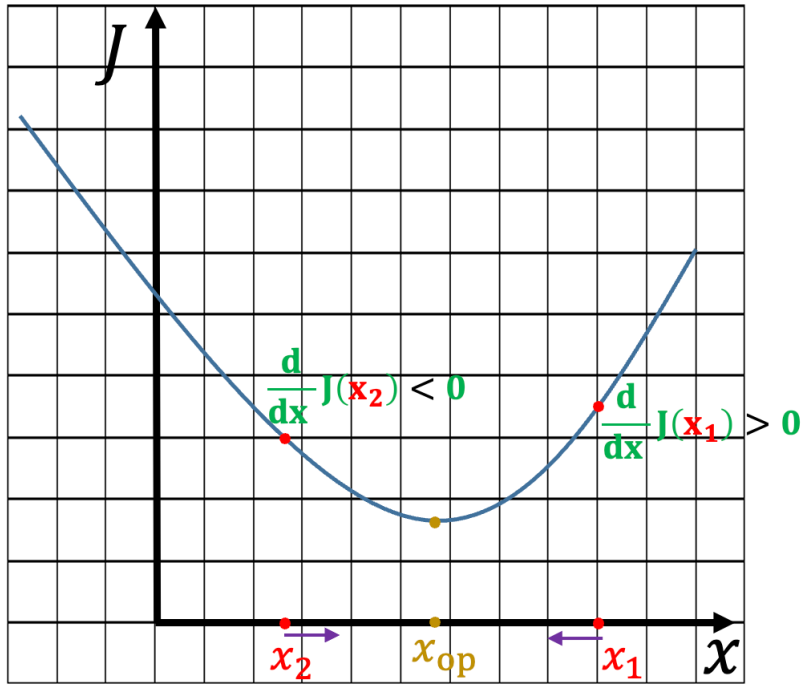
An Optimization Approach

Quang-Vinh Dinh
PhD in Computer Science

Proofreaders: Cao Bá Hoàng & Lê Văn Duy - QA/QC: Nguyễn Thông

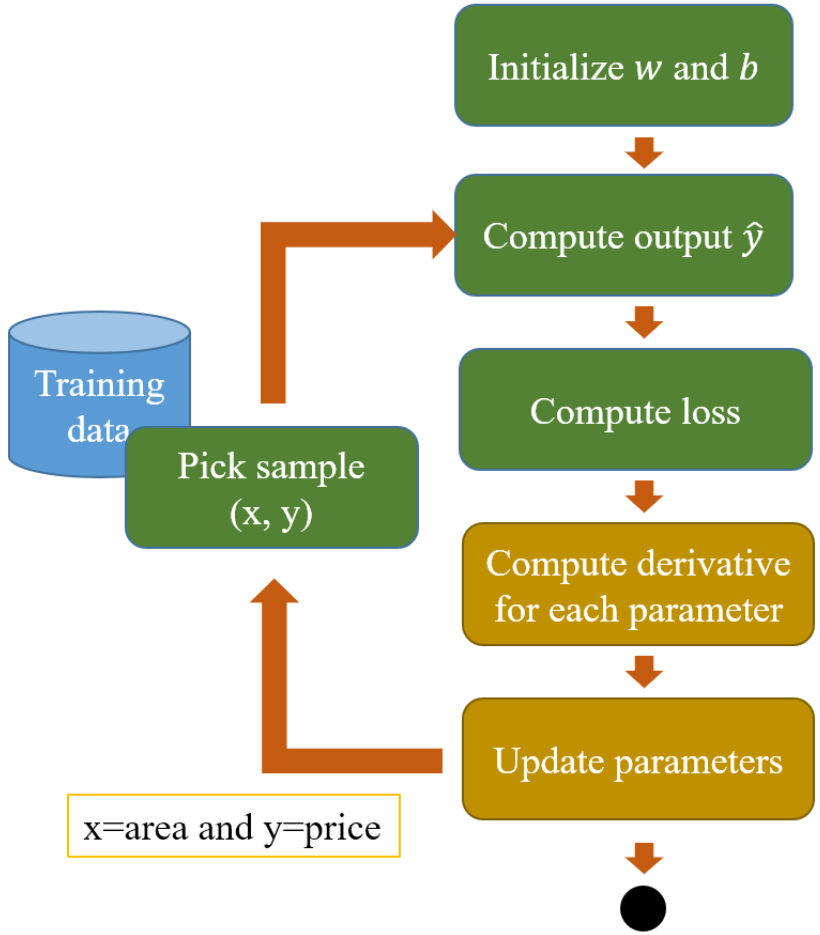
Objectives

Gradient descent



$$x_{new} = x_{old} - \eta \frac{d}{dx} J(x_{old})$$

Linear Regression



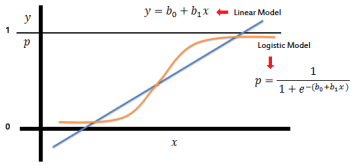
Vectorization

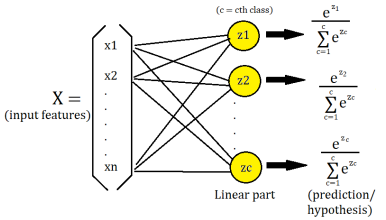
$$\begin{cases} b = b - \eta \dfrac{\partial L}{\partial b} \\ w = w - \eta \dfrac{\partial L}{\partial w} \end{cases}$$

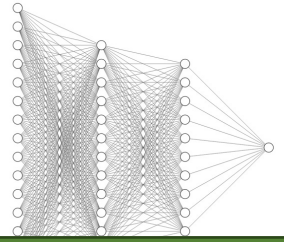
$$\theta \quad \theta \quad \nabla_\theta L$$

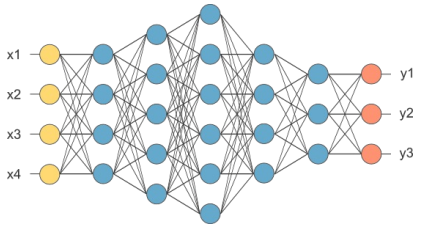
$$\rightarrow \quad \theta = \theta - \eta \nabla_\theta L$$

Motivation

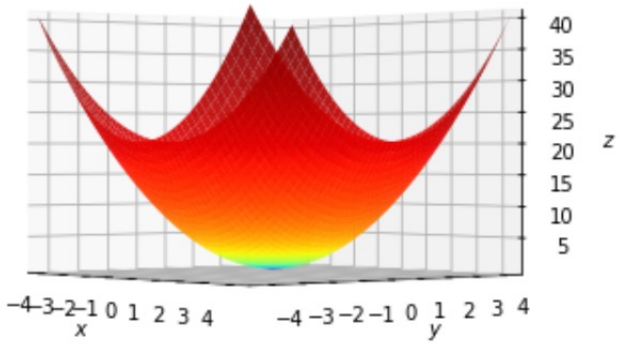


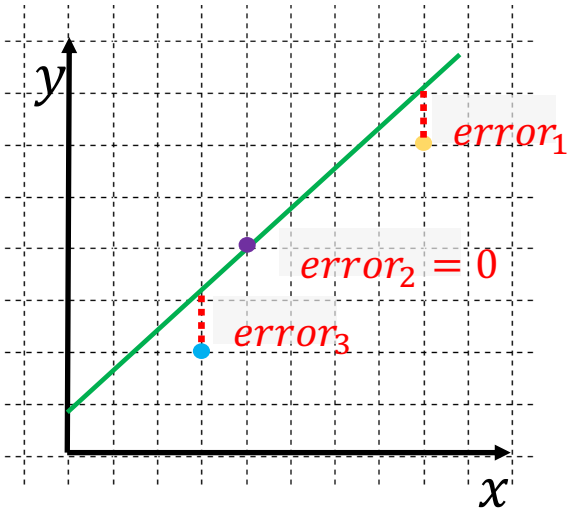






Linear Regression → Logistic Regression → Softmax Regression → Multilayer Perceptron → Deep Learning





1) Pick a sample $(\mathbf{x}, y)$ from training data

2) Compute output $\hat{y}$

$$\hat{y} = \boldsymbol{\theta}^T \boldsymbol{x} = \boldsymbol{x}^T \boldsymbol{\theta}$$

3) Compute loss

$$L = (\hat{y} - y)^2$$

4) Compute derivative

$$\nabla_{\boldsymbol{\theta}} \mathrm{L} = 2\boldsymbol{x}(\hat{y} - y)$$

5) Update parameters

$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \mathrm{L}$$

Outline

SECTION 1

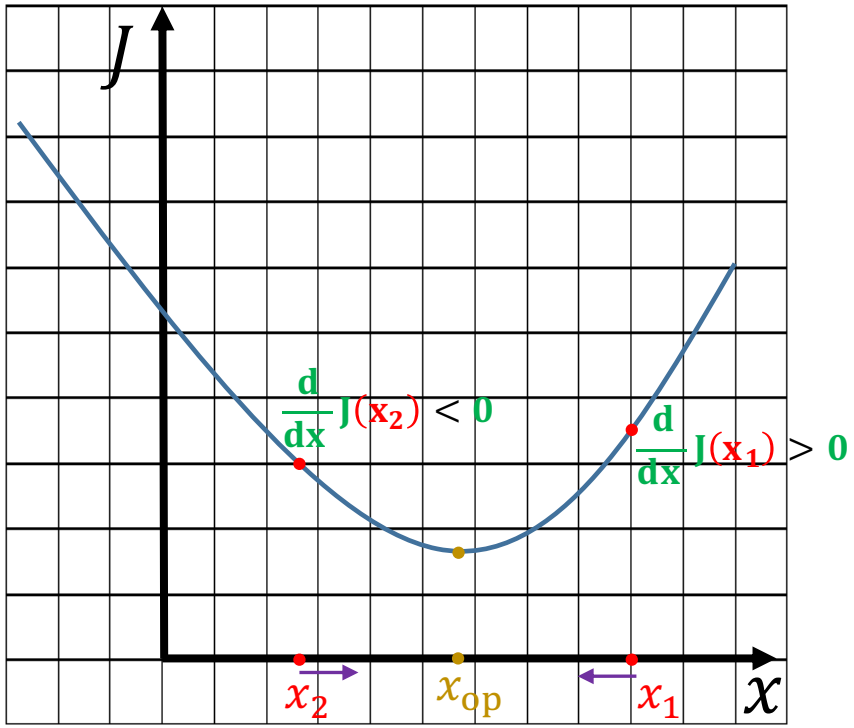
Gradient descent

SECTION 2

Linear Regression

SECTION 3

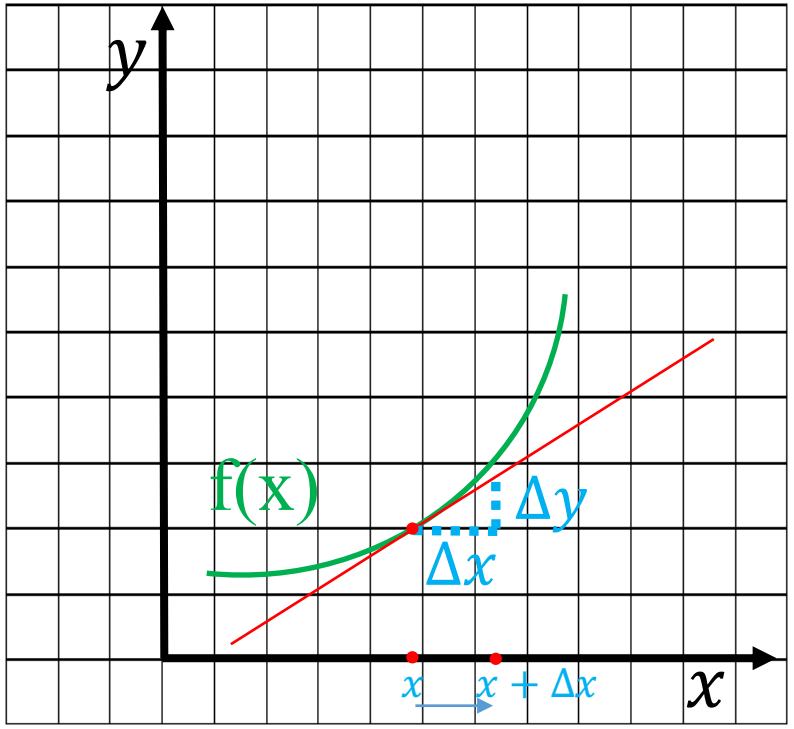
Vectorization



$$x_{new} = x_{old} - \eta \frac{d}{dx} J(x_{old})$$

Derivative

❖ Đạo hàm cho hàm liên tục



$$\frac{d}{dx}f(x), \frac{dy}{dx}, y', f'(x)$$

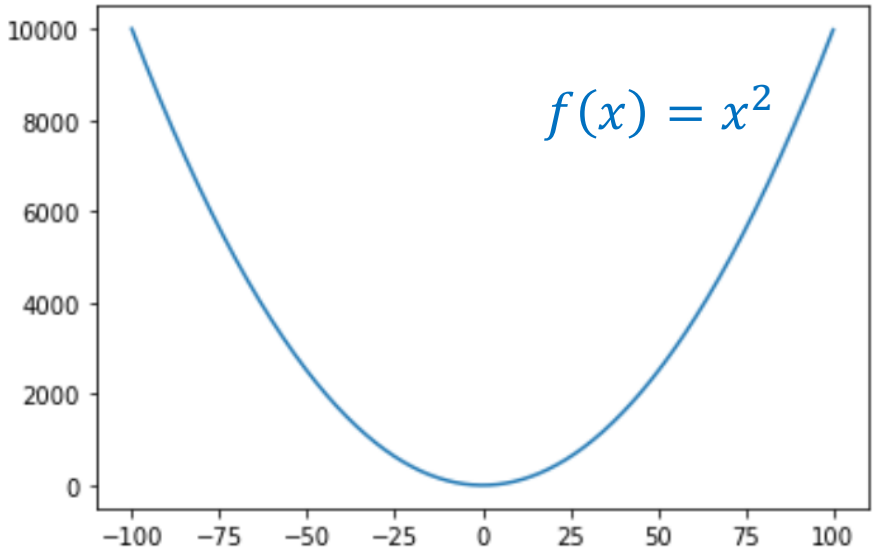
$$\text{Đạo hàm} = \frac{\text{Thay đổi theo } y}{\text{Thay đổi theo } x} = \frac{\Delta y}{\Delta x}$$

$$\frac{d}{dx}f(x) = \lim_{\Delta x \to 0} \frac{f(x+\Delta x) - f(x)}{\Delta x}$$

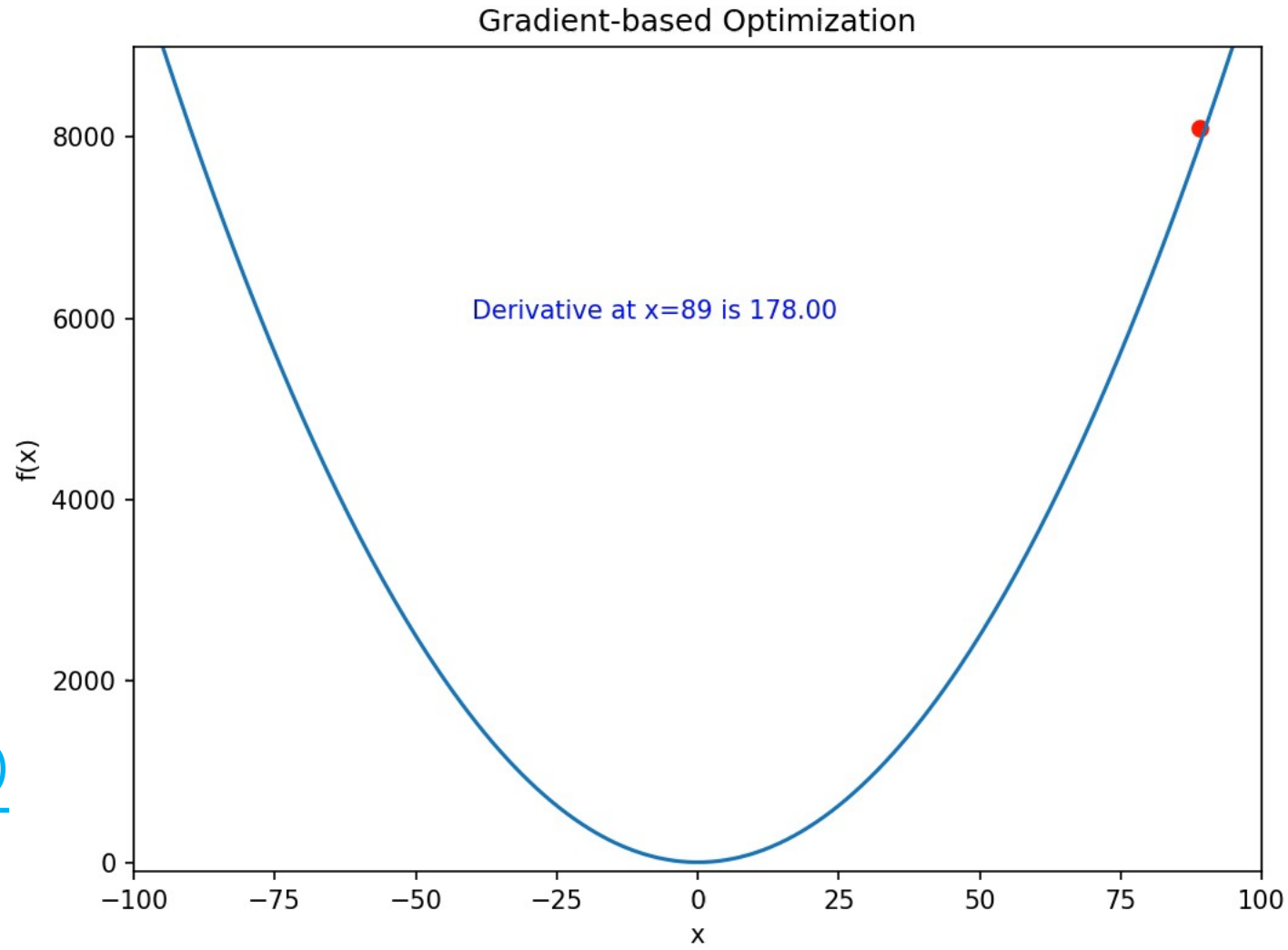
Δx cần tiến về 0 để đường tiếp tuyến tiến về hàm f(x) trong vùng lân cận tại x

Derivative and Optimization

❖ Square function

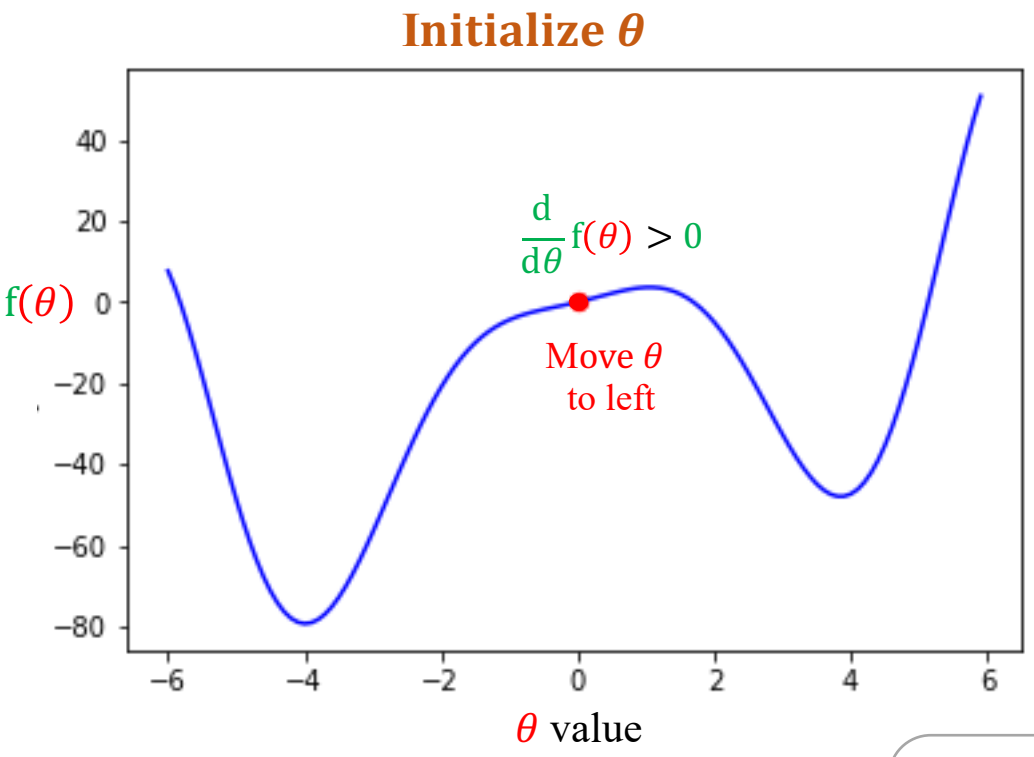


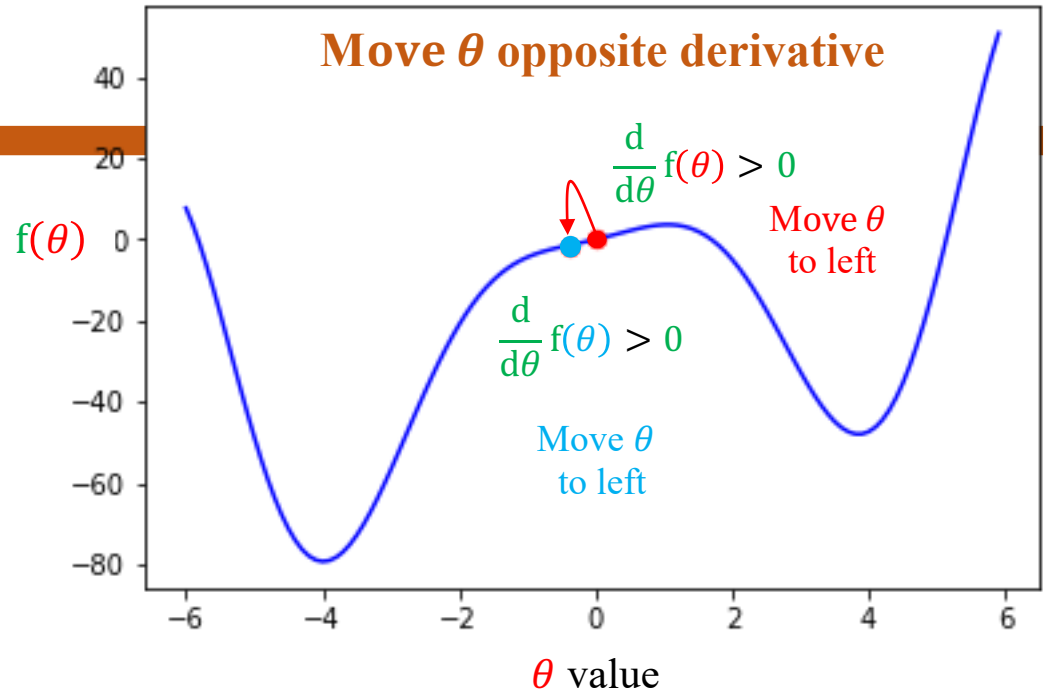
$$\frac{d}{dx} f(x) = \lim_{\Delta x \to 0} \frac{f(x + \Delta x) - f(x)}{\Delta x}$$



Derivative and Optimization

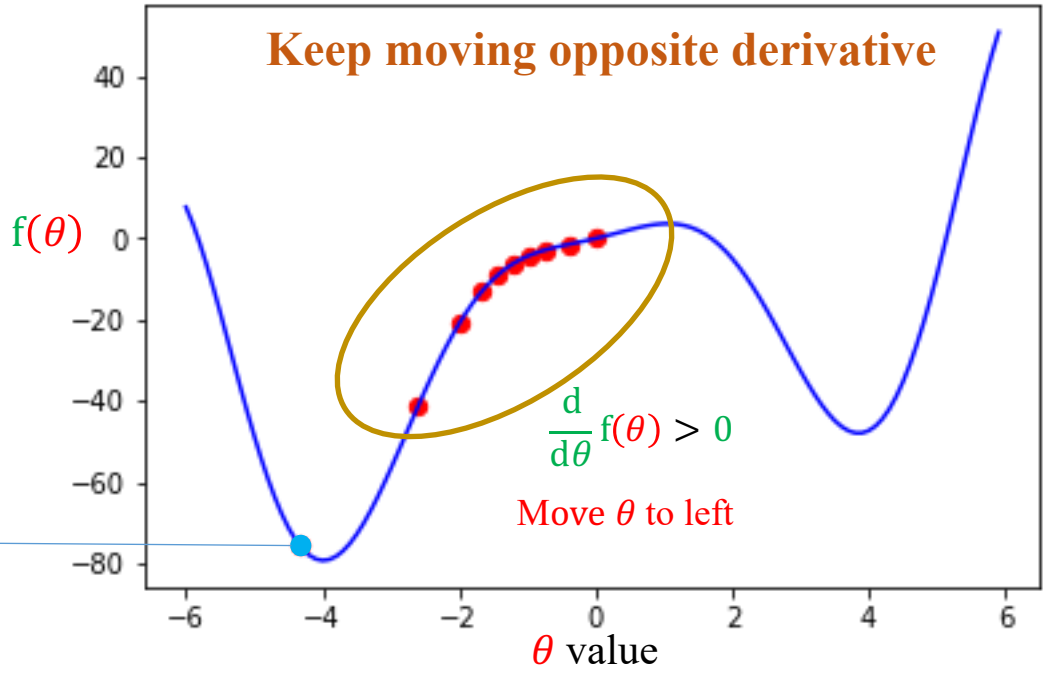
- Test an idea
  - Moving opposite of derivative direction





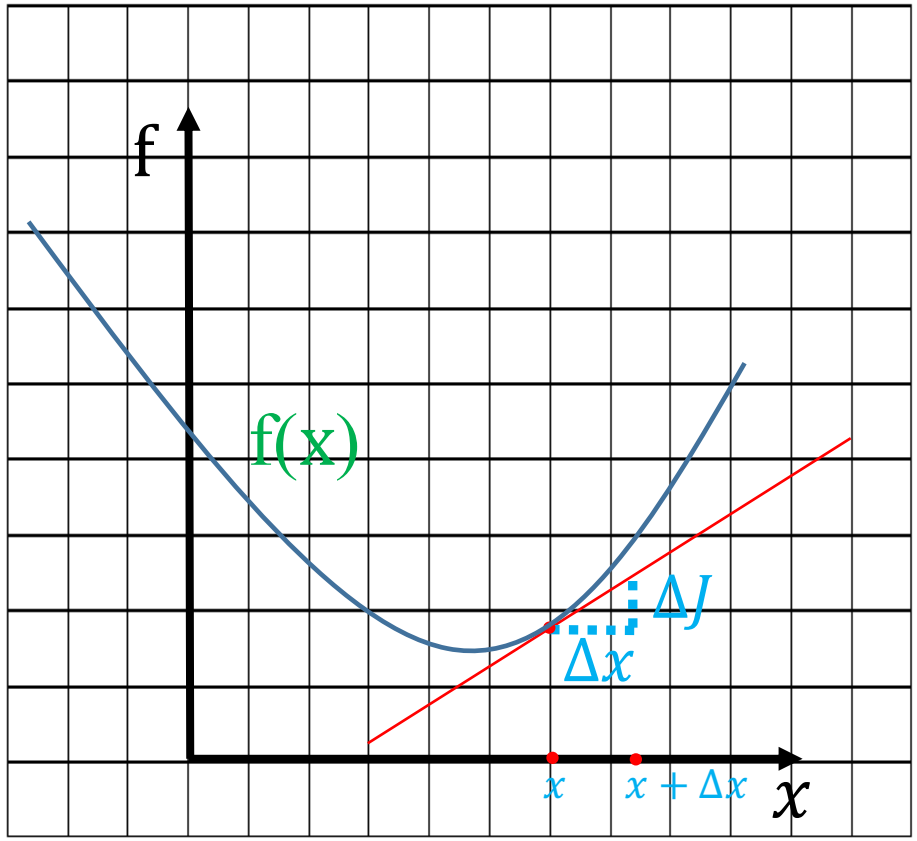
$\frac{d}{d\theta} f(\theta) < 0$

Move θ to right

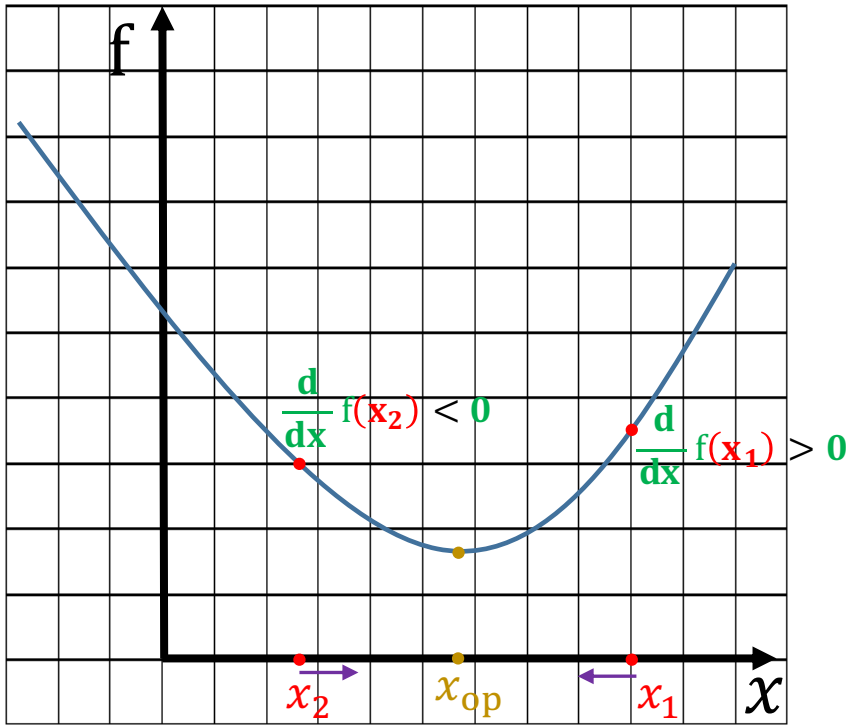


Derivative and Optimization

❖ Gradient descent



$$\frac{d}{dx} f(x) = \lim_{\Delta x \to 0} \frac{f(x + \Delta x) - f(x)}{\Delta x}$$



$$x_{new} = x_{old} - \eta \frac{d}{dx} f(x_{old})$$

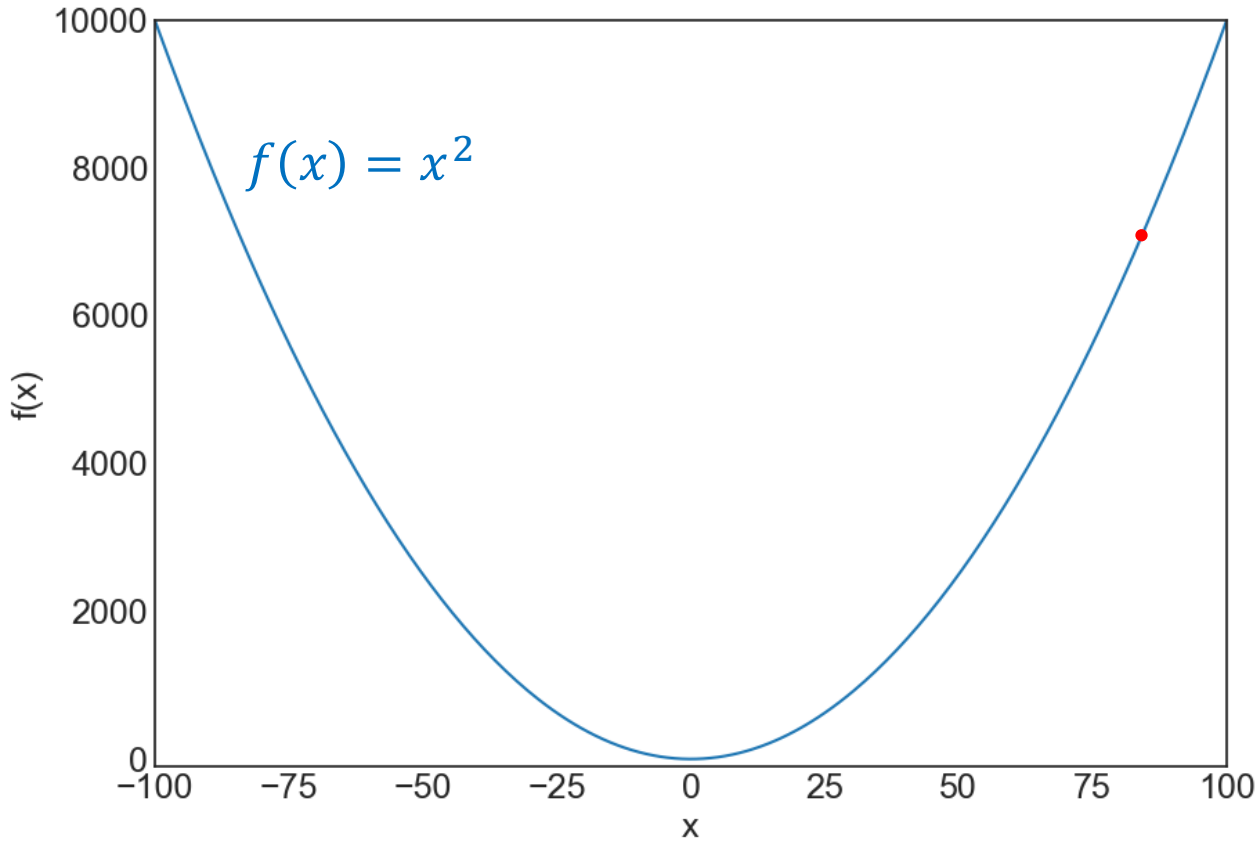
Derivative at x_{old}

learning rate

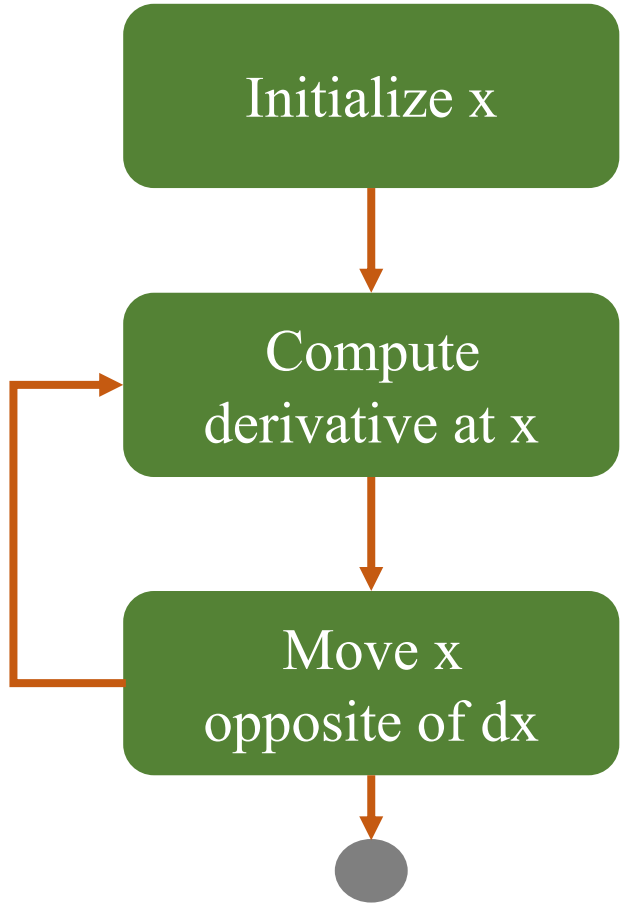
Observation 1

Optimization

❖ Square function



$$x_t = x_{t-1} - \eta f'(x_{t-1})$$



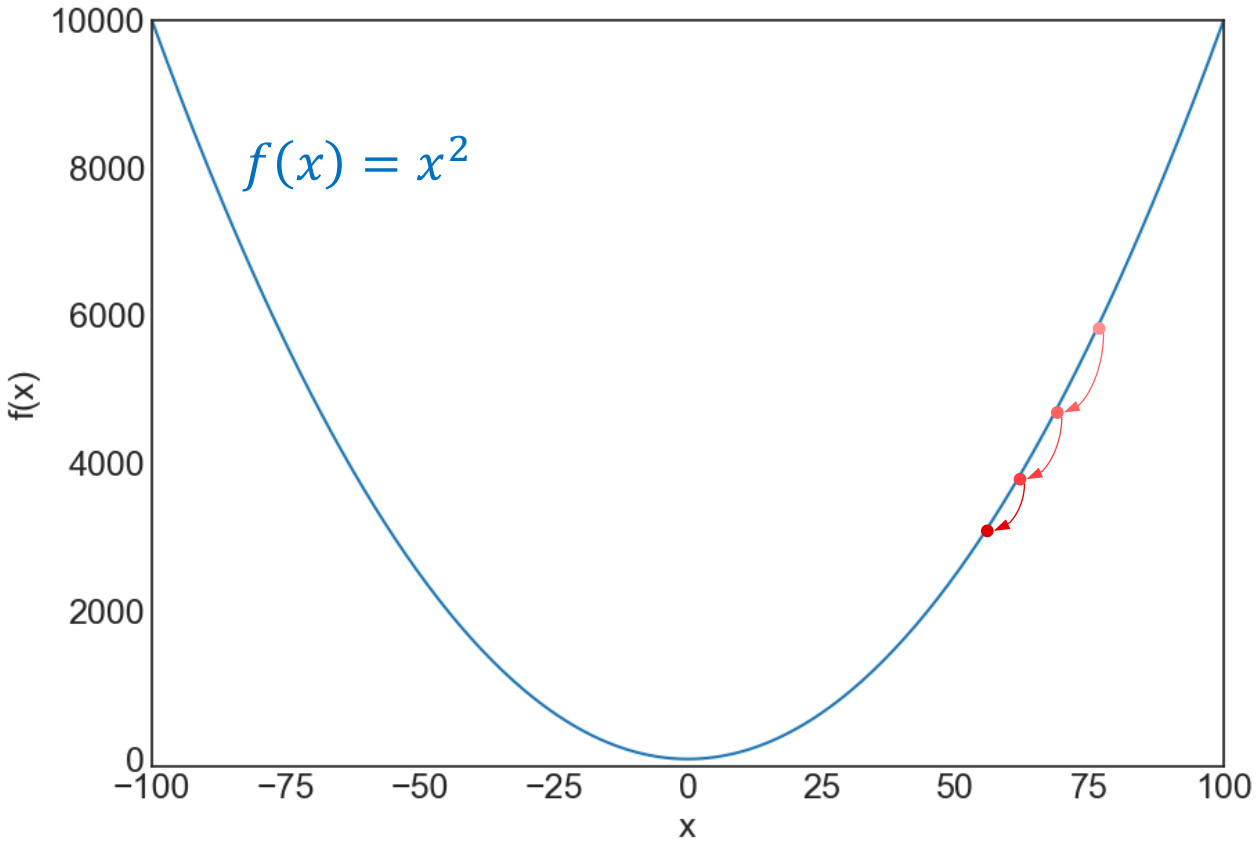
$$-100 \leq x \leq 100$$

$$x \in \mathbb{R}$$

Optimization

$$x_t = x_{t-1} - \eta f'(x_{t-1})$$

❖ Square function



$$-100 \leq x \leq 100$$

$$x \in \mathbb{R}$$

$x_0 = 70.0 \qquad \eta = 0.1$

$f'(x_0) = 140.0$

$x_1 = x_0 - \eta f'(x_0) = 56.0$

$f'(x_1) = 112.0$

$x_2 = x_1 - \eta f'(x_1) = 44.8$

$f'(x_2) = 89.6$

$x_3 = x_2 - \eta f'(x_2) = 35.84$

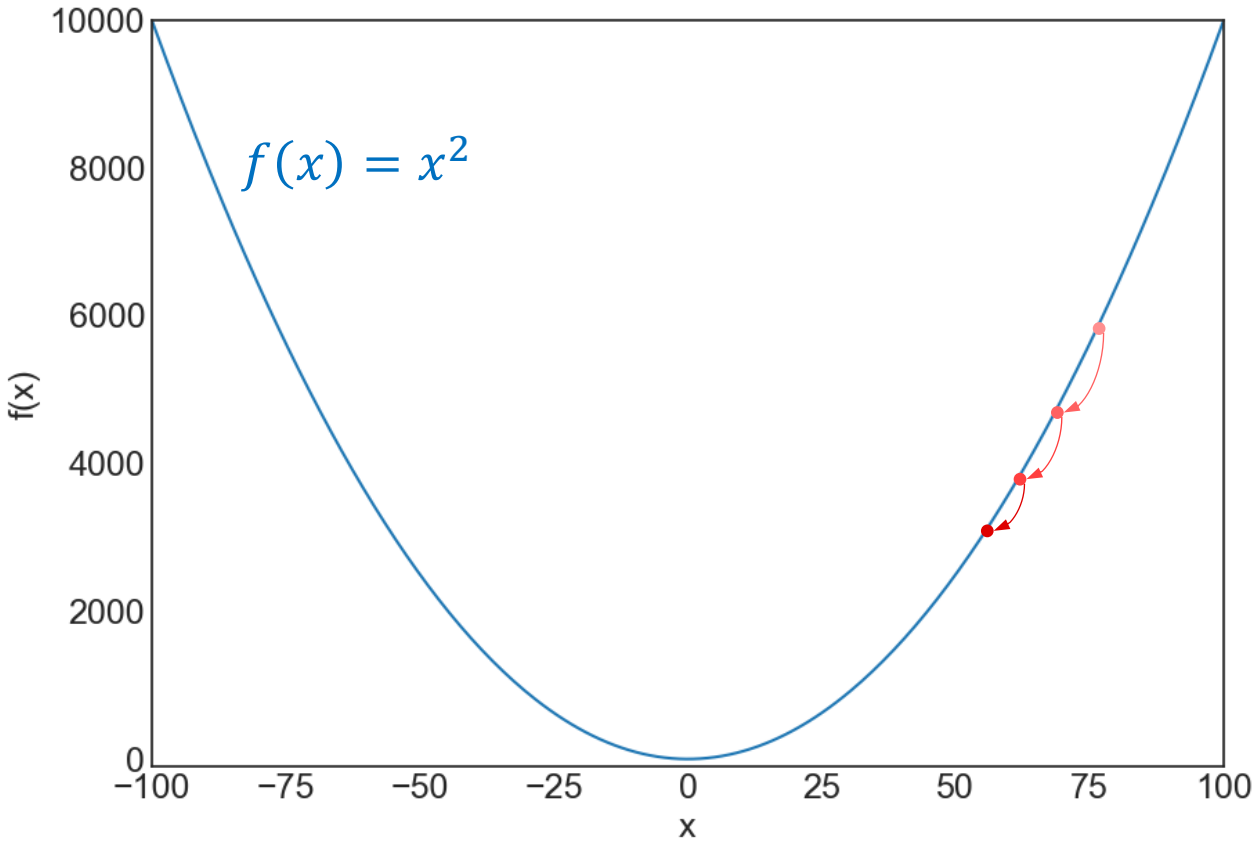
$f'(x_3) = 71.68$

$x_4 = x_3 - \eta f'(x_3) = 28.672$

Optimization

$$x_t = x_{t-1} - \eta f'(x_{t-1})$$

❖ Square function



$x_{30} = 0.069 \qquad \eta = 0.1$

$f'(x_{30}) = 0.138$

$x_{31} = x_{30} - \eta f'(x_{30}) = 0.055$

$f'(x_{31}) = 0.11$

$x_{32} = x_{31} - \eta f'(x_{31}) = 0.044$

$f'(x_{32}) = 0.088$

$x_{33} = x_{32} - \eta f'(x_{32}) = 0.035$

$f'(x_{34}) = 0.071$

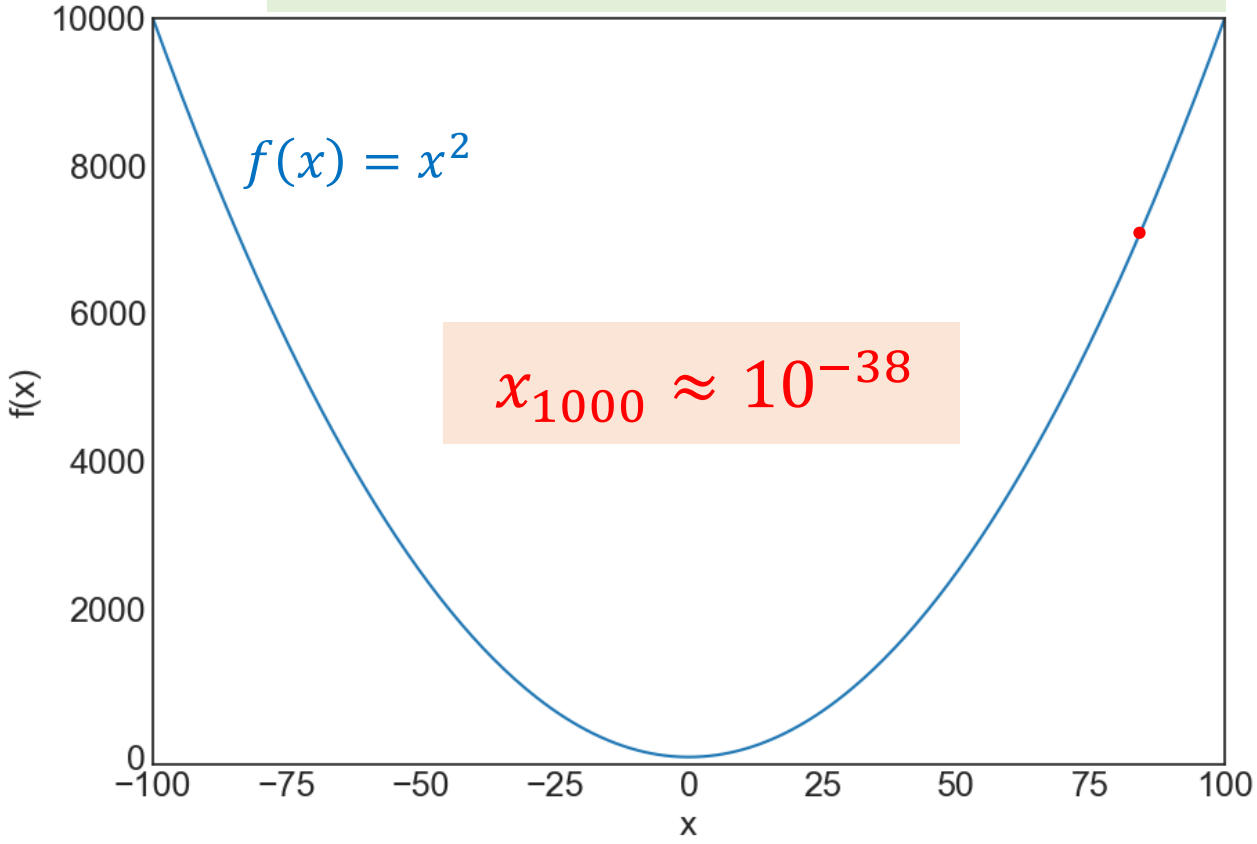
$x_{34} = x_{33} - \eta f'(x_{33}) = 0.028$

Keep doing

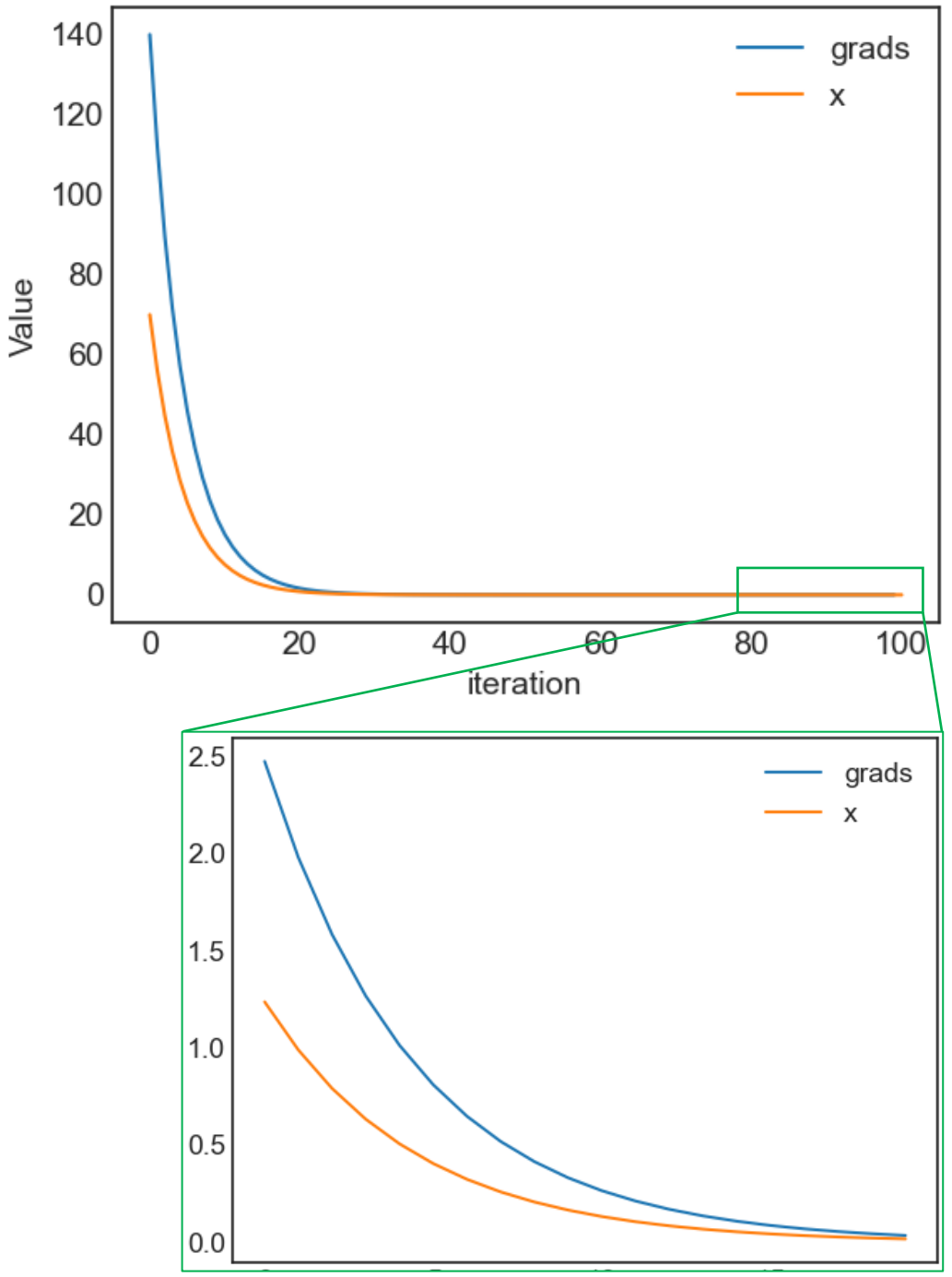
Optimization

❖ Square function

$$x_t = x_{t-1} - \eta f'(x)$$



Optimized successfully!



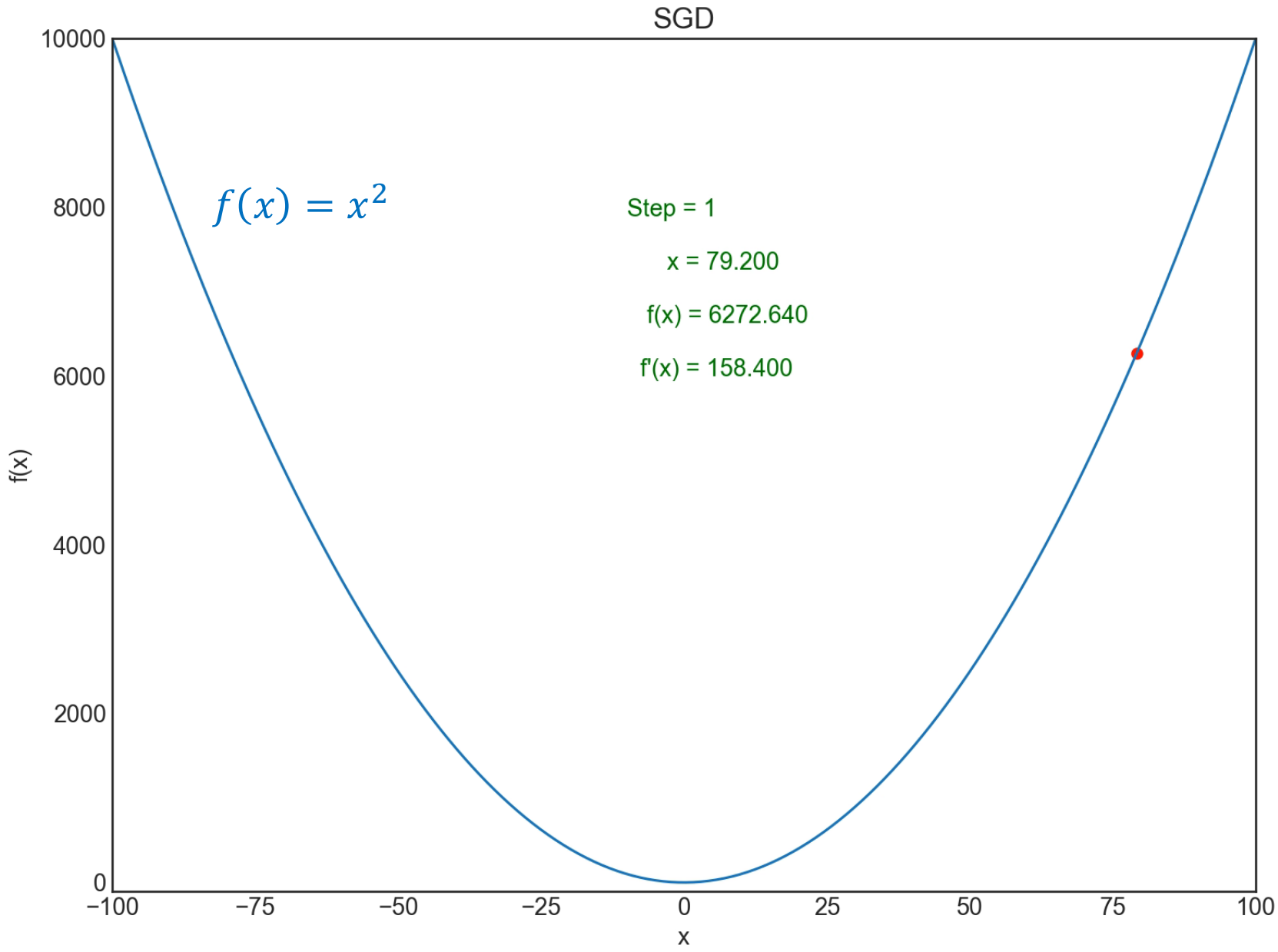
Optimization

❖ **Square function**

$x_0 = 99.0$

$\eta = 0.1$

$$x_t = x_{t-1} - \eta f'(x)$$



Optimization

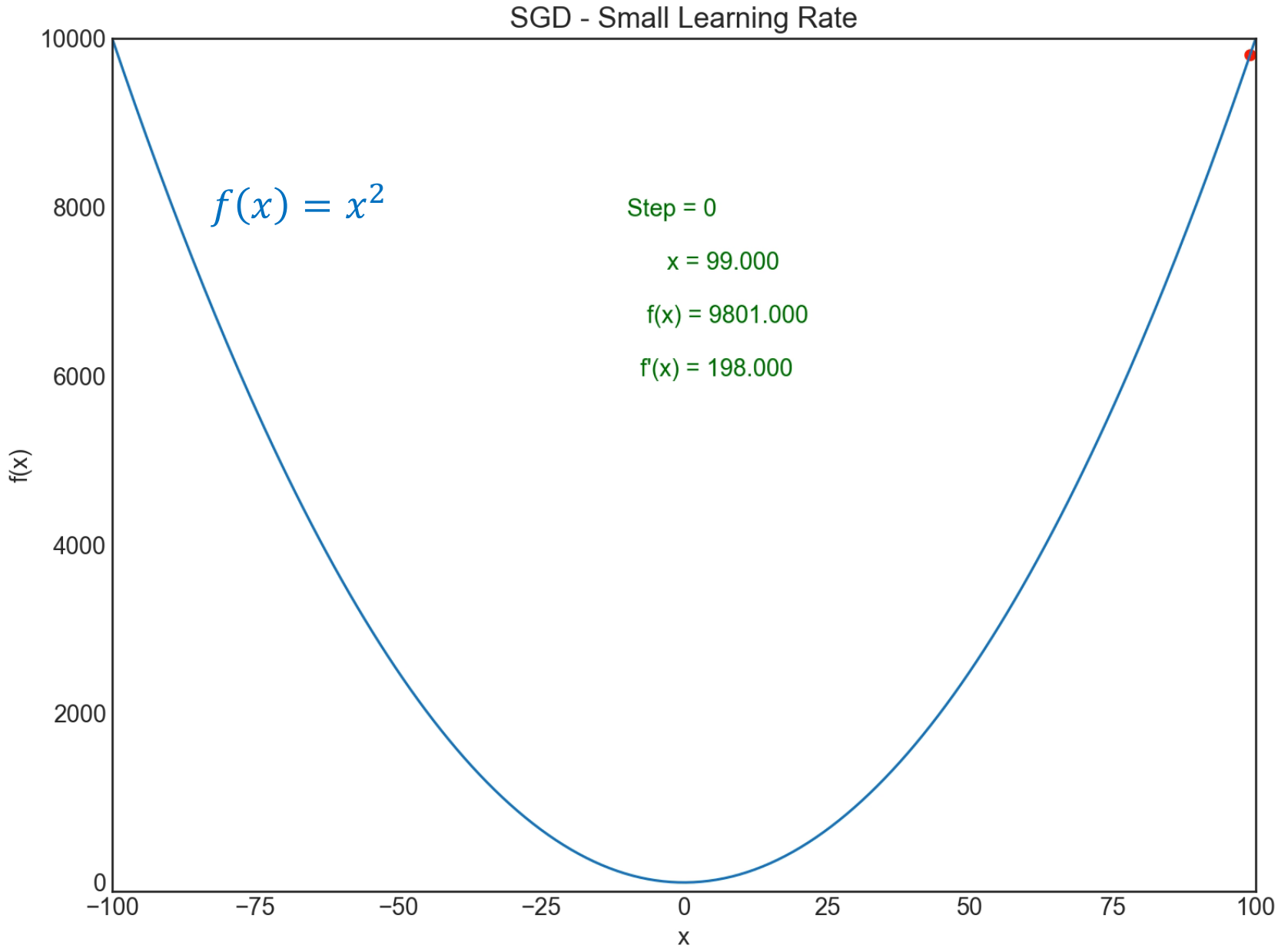
❖ **Square function**

Discussion

$x_0 = 99.0$

$\eta = 0.001$

$$x_t = x_{t-1} - \eta f'(x)$$



Optimization

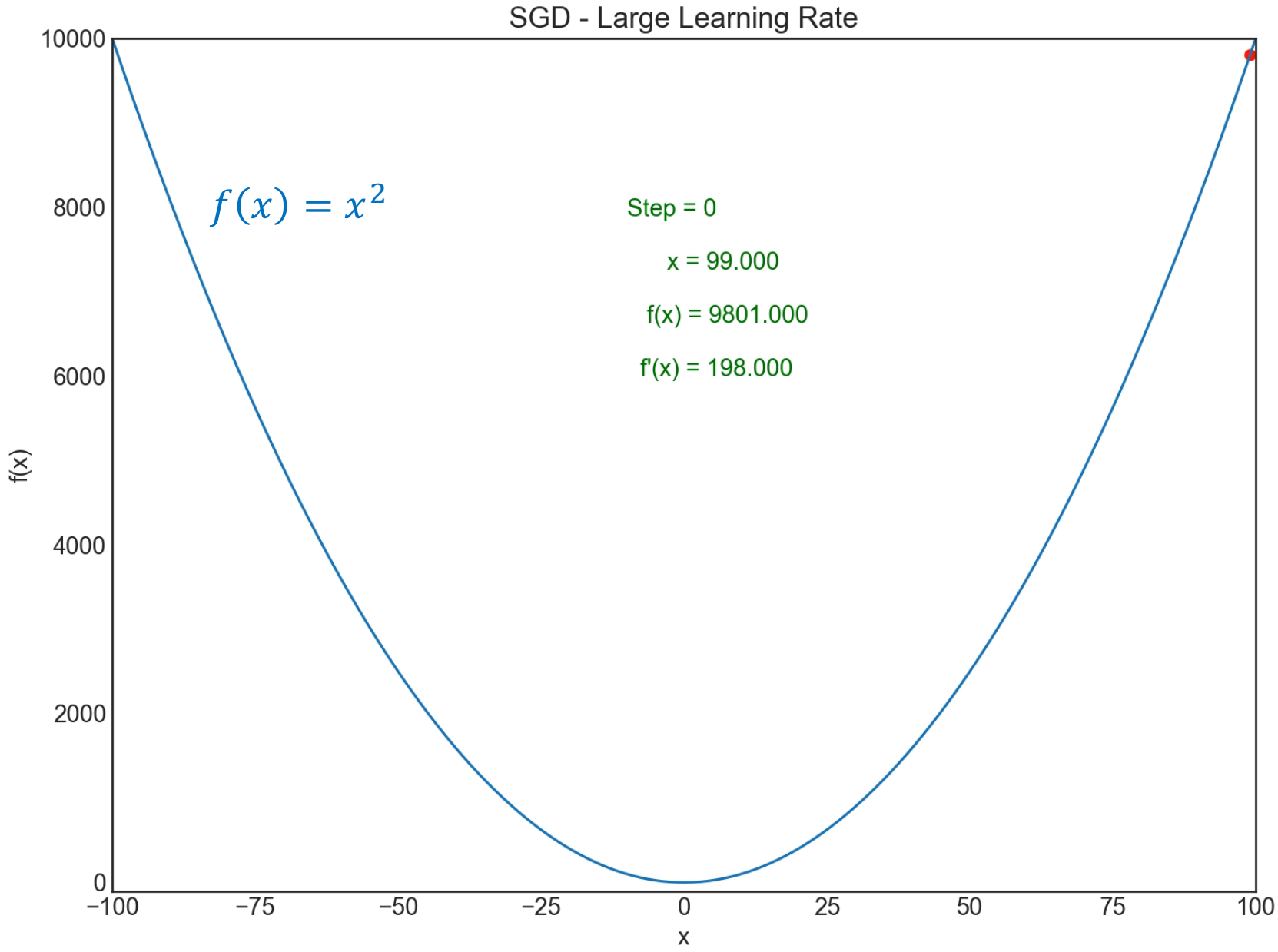
❖ **Square function**

Discussion

$x_0 = 99.0$

$\eta = 0.8$

$$x_t = x_{t-1} - \eta f'(x)$$



Optimization

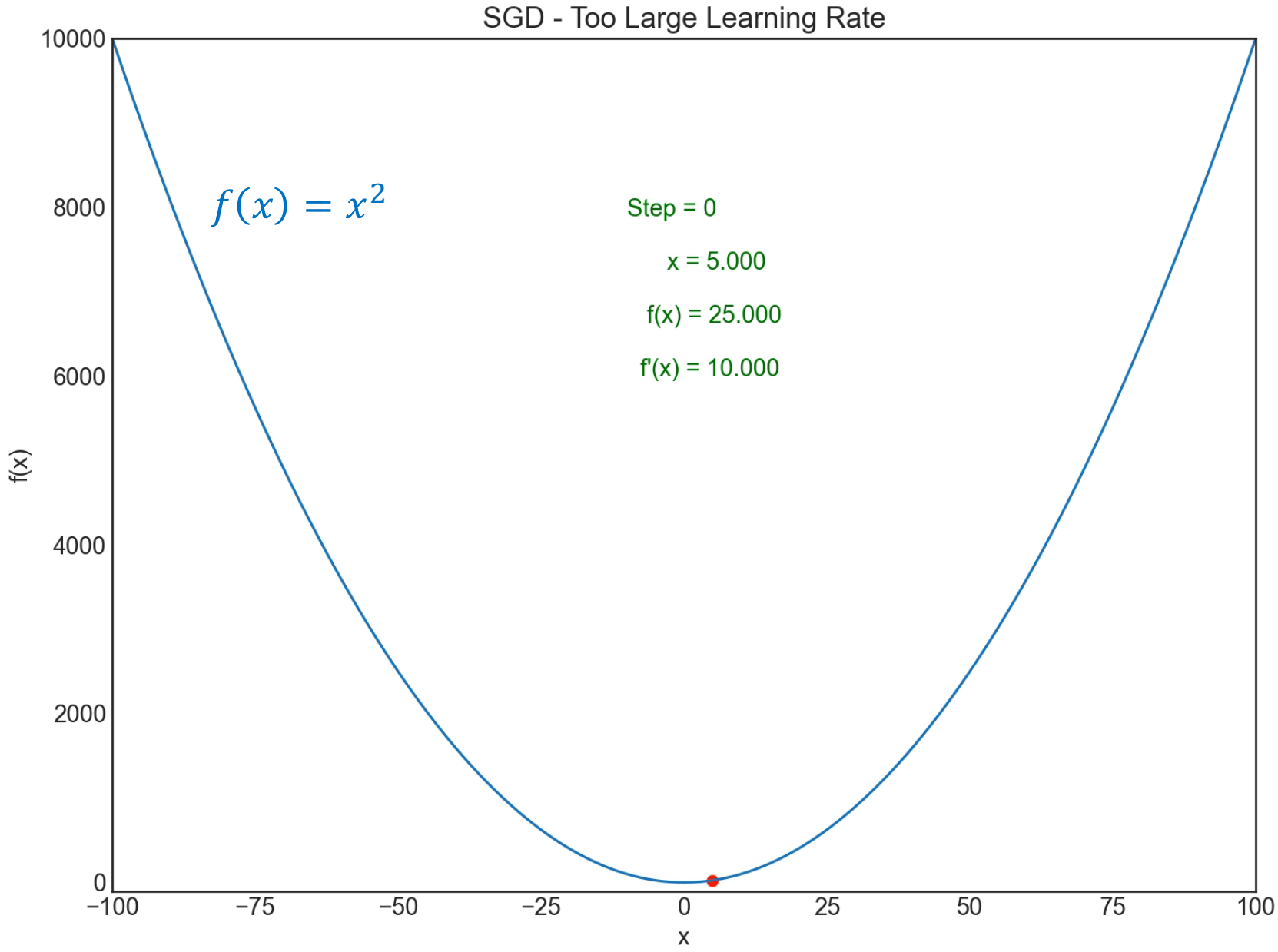
❖ **Square function**

Discussion

$x_0 = 99.0$

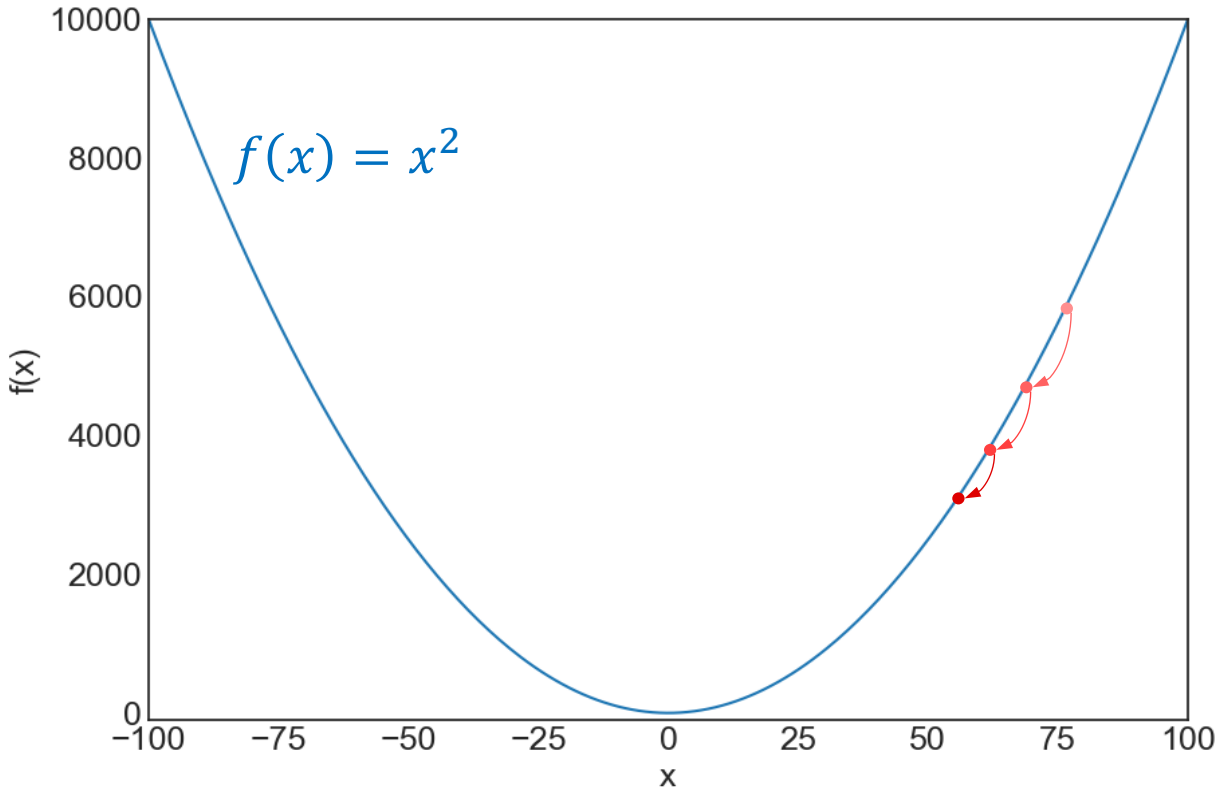
$\eta = 1.1$

$$x_t = x_{t-1} - \eta f'(x)$$



Optimization

❖ Square function: Summary



$$x_t = x_{t-1} - \eta f'(x_{t-1})$$

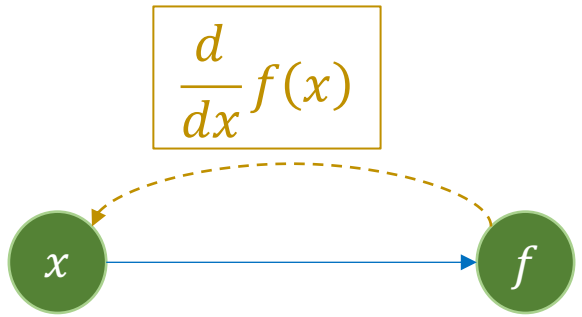
$x_{30} = 0.069 \qquad \eta = 0.1$

$f'(x_{30}) = 0.138$

$x_{31} = x_{30} - \eta f'(x_{30}) = 0.055$

$f'(x_{31}) = 0.11$

$x_{32} = x_{31} - \eta f'(x_{31}) = 0.044$



What does f(x) mean?

- Given a function f(x), find optimal x_{opt} so that $f(x_{opt})$ is minimum
- After an update, $f(x_{new}) \leq f(x_{old})$

Observation 2

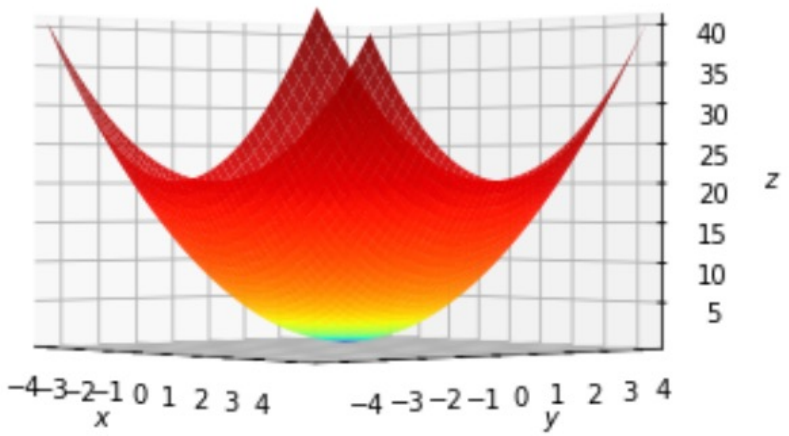
Optimization

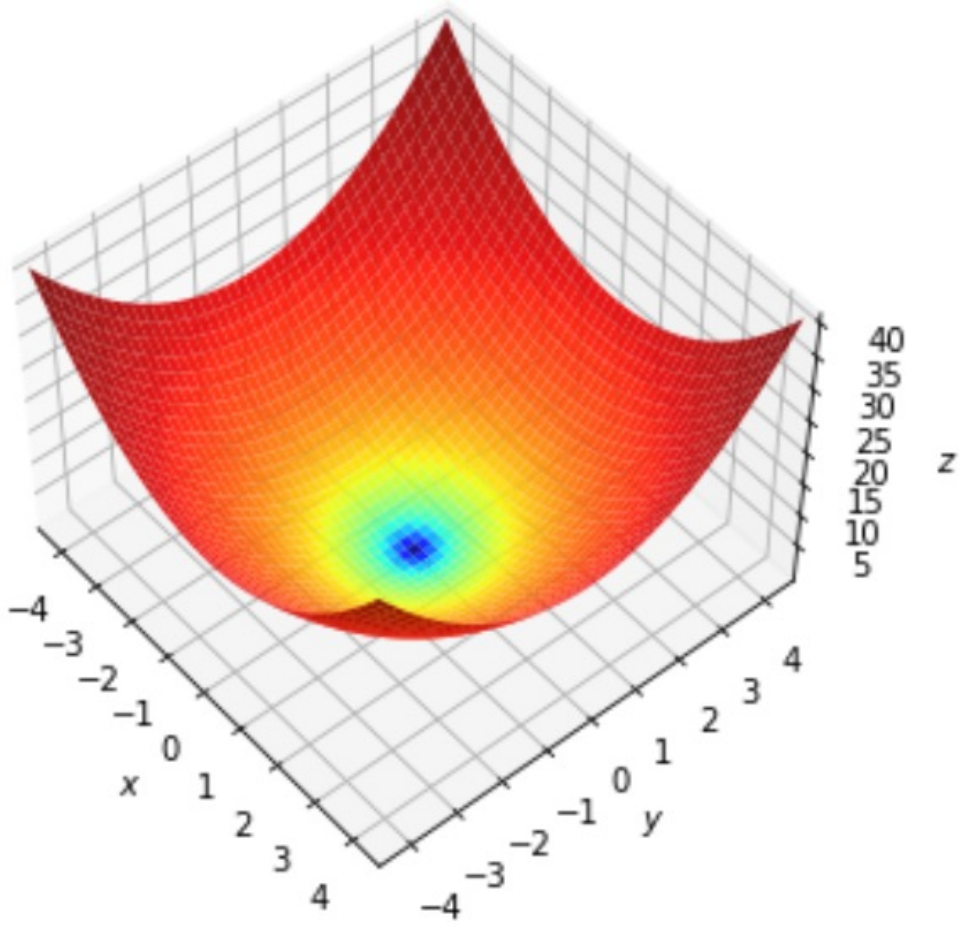
❖ Optimization: 2D function

$$f(x, y) = x^2 + y^2$$

$$-100 \leq x, y \leq 100$$

$$x, y \in \mathbb{N}$$





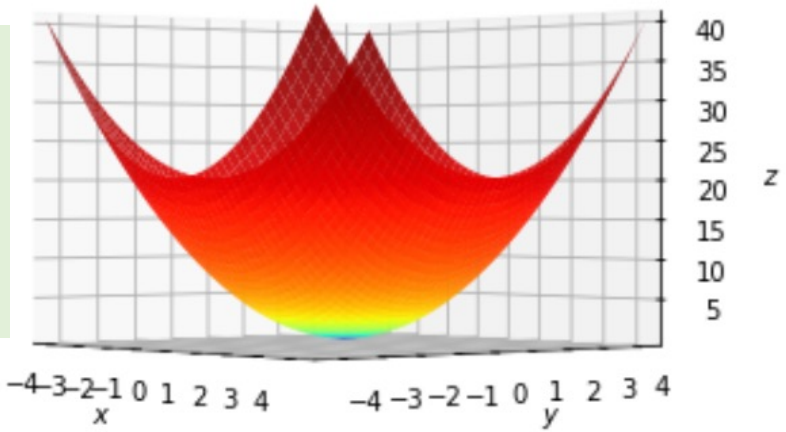
Optimization

❖ Optimization: 2D function

$$f(x,y) = x^2 + y^2$$

$$-100 \le x, y \le 100$$

$$x, y \in \mathbb{N}$$



$$x = x - \eta \frac{\partial f(x,y)}{\partial x}$$

$$y = y - \eta \frac{\partial f(x,y)}{\partial y}$$

$x_0 = 6.0 \qquad y_0 = 9.0 \qquad \eta = 0.1$

$\frac{\partial f(x_0,y_0)}{\partial x} = 12 \qquad \frac{\partial f(x_0,y_0)}{\partial y} = 18$

$x_1 = 4.8 \qquad y_1 = 7.2$

$\frac{\partial f(x_1,y_1)}{\partial x} = 9.6 \qquad \frac{\partial f(x_1,y_1)}{\partial y} = 14.4$

$x_2 = 3.84 \qquad y_2 = 5.75$

$\frac{\partial f(x_2,y_2)}{\partial x} = 7.68 \qquad \frac{\partial f(x_2,y_2)}{\partial y} = 11.51$

$x_3 = 3.07 \qquad y_3 = 4.608$

$\frac{\partial f(x_3,y_3)}{\partial x} = 6.14 \qquad \frac{\partial f(x_3,y_3)}{\partial y} = 9.21$

$x_4 = 2.45 \qquad y_4 = 3.68$

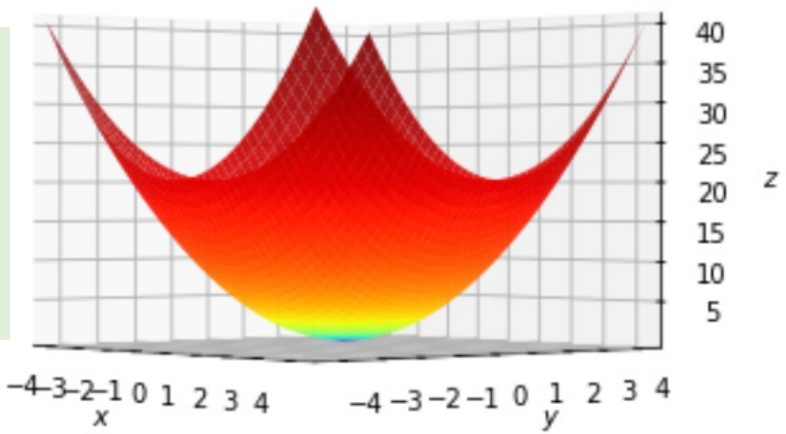
Optimization

❖ Optimization: 2D function

$$f(x,y) = x^2 + y^2$$

$$-100 \le x, y \le 100$$

$$x, y \in \mathbb{N}$$



$$\frac{\partial f(x_2,y_2)}{\partial x} = 7.68 \qquad \frac{\partial f(x_2,y_2)}{\partial y} = 11.51$$

$$x_3 = 3.07 \qquad y_3 = 4.608$$

$$\frac{\partial f(x_3,y_3)}{\partial x} = 6.14 \qquad \frac{\partial f(x_3,y_3)}{\partial y} = 9.21$$

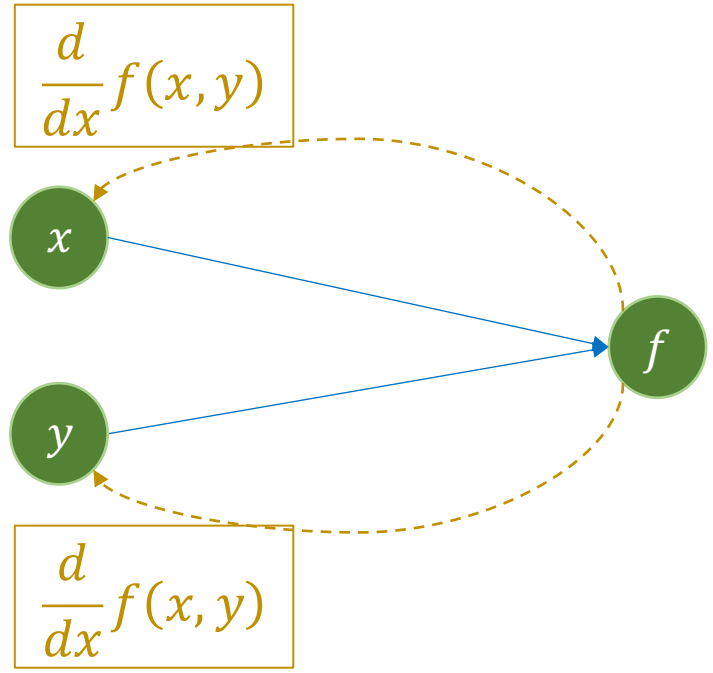
$$x_4 = 2.45 \qquad y_4 = 3.68$$

Summary:

- Given a function f(x, y), find optimal (x_{opt}, y_{opt}) so that $f(x_{opt}, y_{opt})$ is minimum

- After an update, $f(x_{new}, y_{new}) \le f(x_{old}, y_{old})$

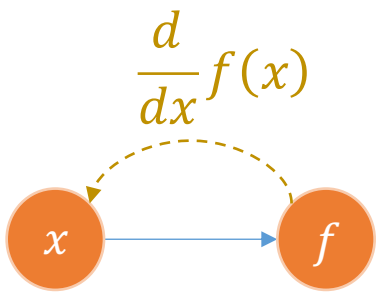
What does f(x,y) mean?

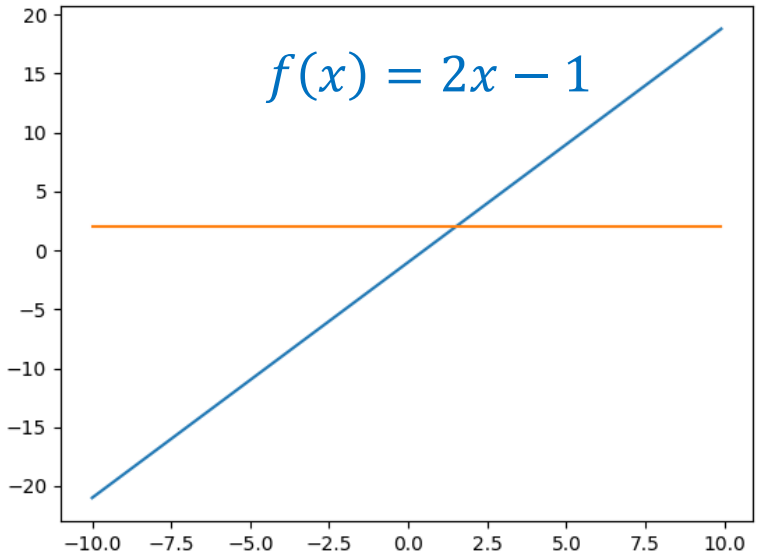


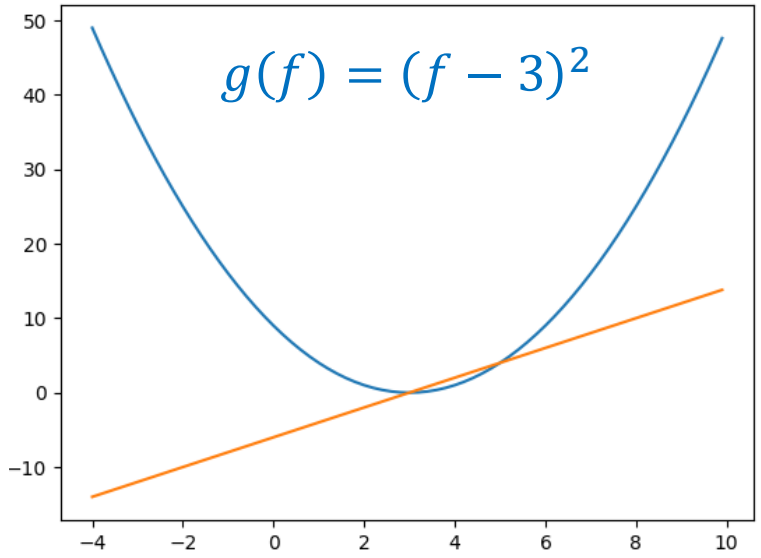
Observation 3

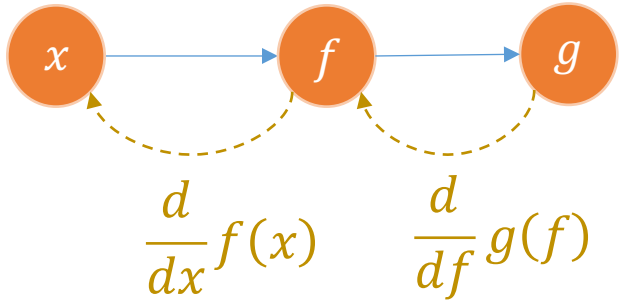
Gradient-based Optimization

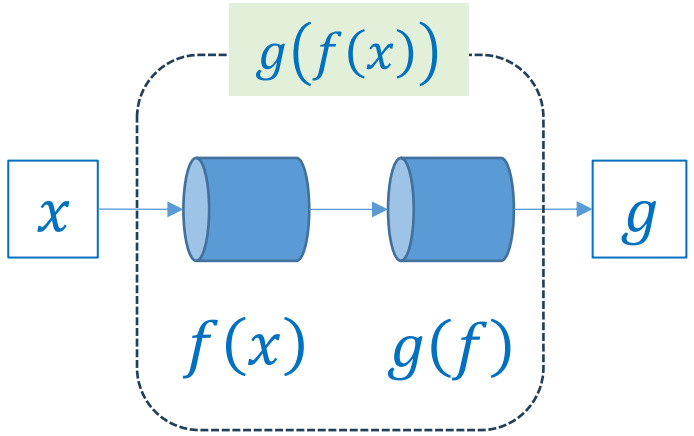
❖ For composite function

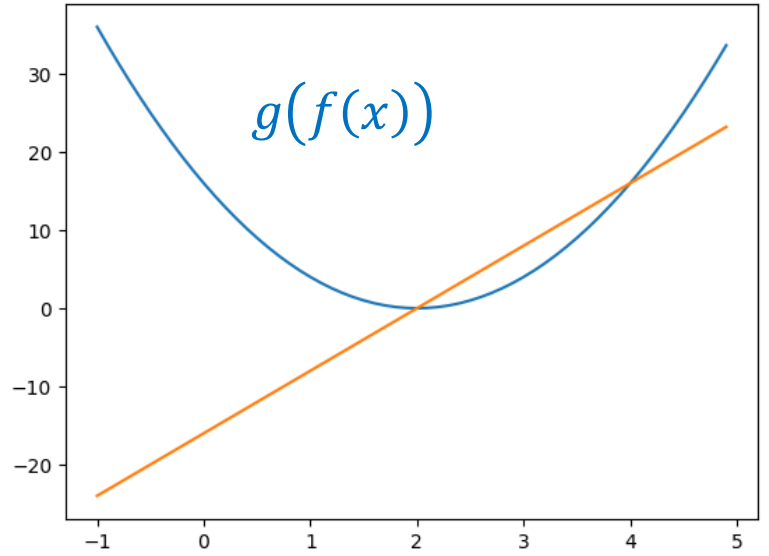








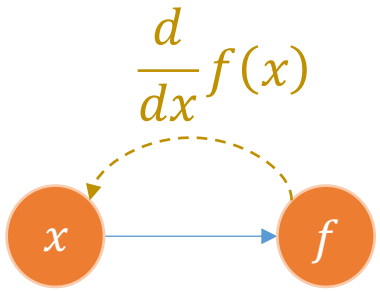


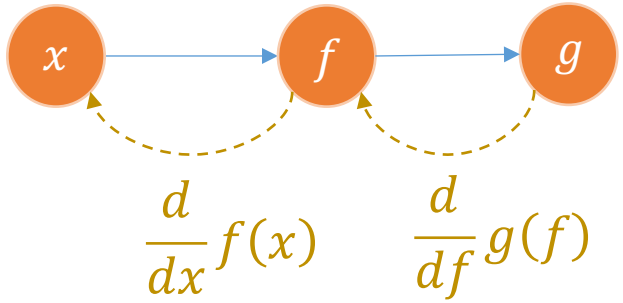


$$\frac{d}{dx}g(f(x)) = \left[\frac{d}{df}g(f)\right] * \left[\frac{d}{dx}f(x)\right]$$

Gradient-based Optimization

❖ For composite function and chain rule





$$\frac{d}{dx}g(f(x)) = \left[\frac{d}{df}g(f)\right] * \left[\frac{d}{dx}f(x)\right]$$

$$f(x) = 2x - 1$$

$$g(f) = (f-3)^2$$

$$f'(x) = 2$$

$$g'(f) = 2(f-3)$$

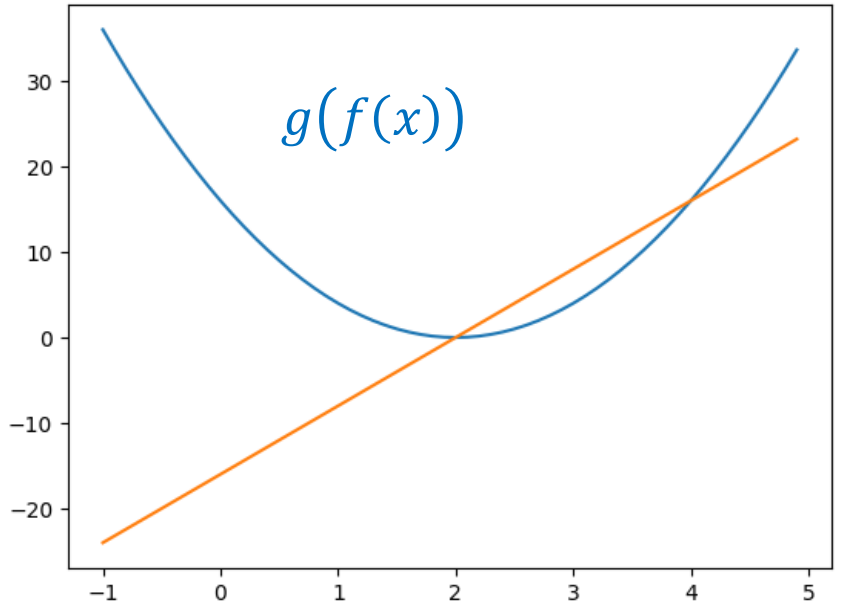
$$\frac{dg}{dx} = \frac{dg}{df}\frac{df}{dx}$$

$$= 2(f-3)2$$

$$= 4(2x-1-3)$$

$$= 8x - 16$$

Implementation



$$f(x) = 2x - 1$$

$$g(f) = (f - 3)^2$$

$$\frac{dg}{dx} = 8x - 16$$

```python
def fx(x):
    return 2*x - 1

def gf(f):
    return (f-3)**2

def dg_dx(x):
    return 8*x - 16
```

```python
import random

# parameters
lr = 0.1

# initialize x
x = 60

old_loss = gf(fx(x)) # Logging
print(f'old_loss: {old_loss}')

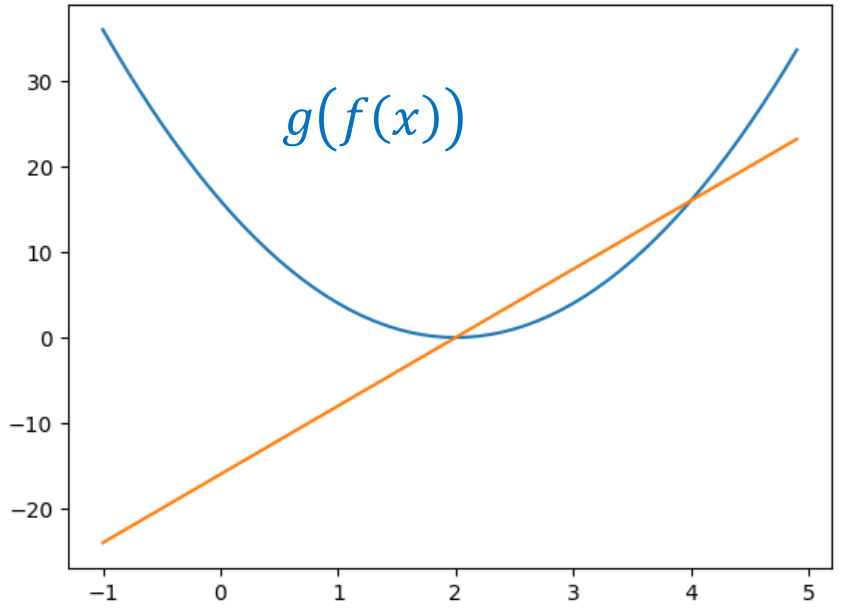
# compute derivative
dg_dx_value = dg_dx(x)

# update
x = x - lr*dg_dx_value

new_loss = gf(fx(x)) # Logging
print(f'new_loss: {new_loss}')
```

```
old_loss: 13456
new_loss: 538.2399999999994
```

Implementation



$$f(x) = 2x - 1$$

$$g(f) = (f - 3)^2$$

$$\frac{dg}{dx} = 8x - 16$$

```python
def fx(x):
    return 2*x - 1

def gf(f):
    return (f-3)**2

def dg_dx(x):
    return 8*x - 16
```

```python
import random

# parameters
num_steps = 5
lr = 0.1

# set x randomly
x = random.randint(-100, 100)

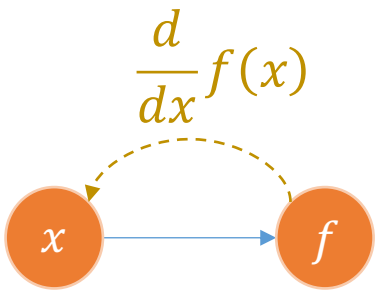
for _ in range(num_steps):
    # logging
    loss = gf(fx(x))

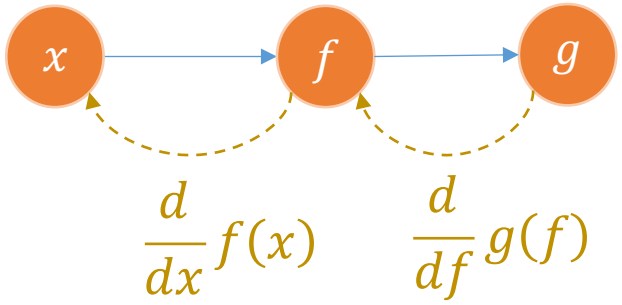
    # compute derivative
    dg_dx_value = dg_dx(x)

    # update
    x = x - lr*dg_dx_value
```

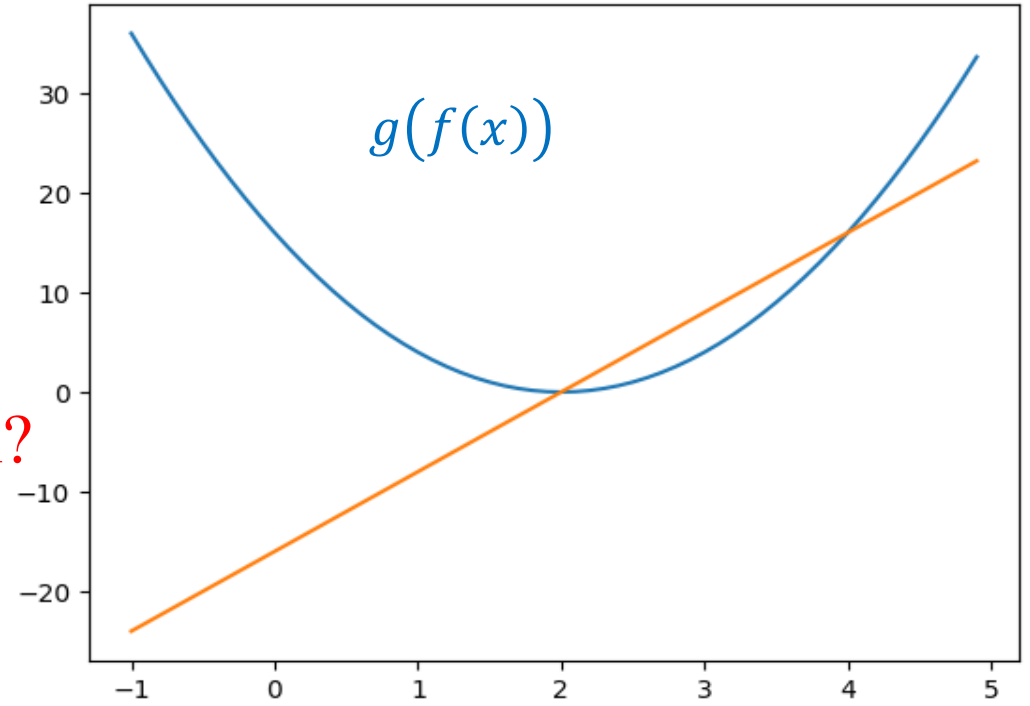
Gradient-based Optimization

❖ For composite function: Summary





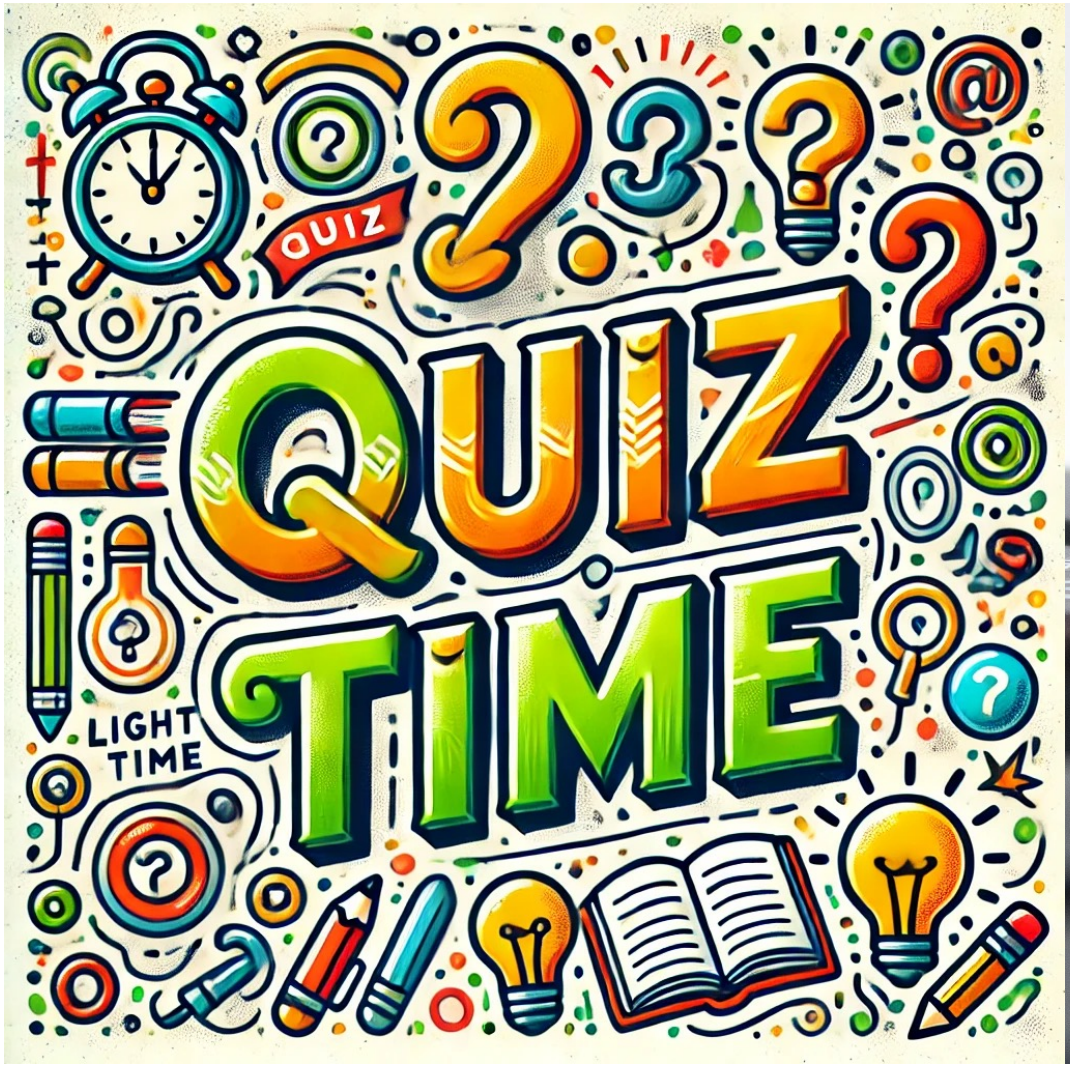
What does g(f(x)) mean?



$$\frac{d}{dx}g(f(x)) = \left[\frac{d}{df}g(f)\right] * \left[\frac{d}{dx}f(x)\right]$$

- Given a function g(f(x)), find optimal x_{opt} so that $g(f(x_{opt}))$ is minimum

- After an update, $g(f(x_{new})) \leq g(f(x_{old}))$


QUIZ
QUIZ
TIME
LIGHT
TIME


BREAK
TIME!!!

Outline

SECTION 1

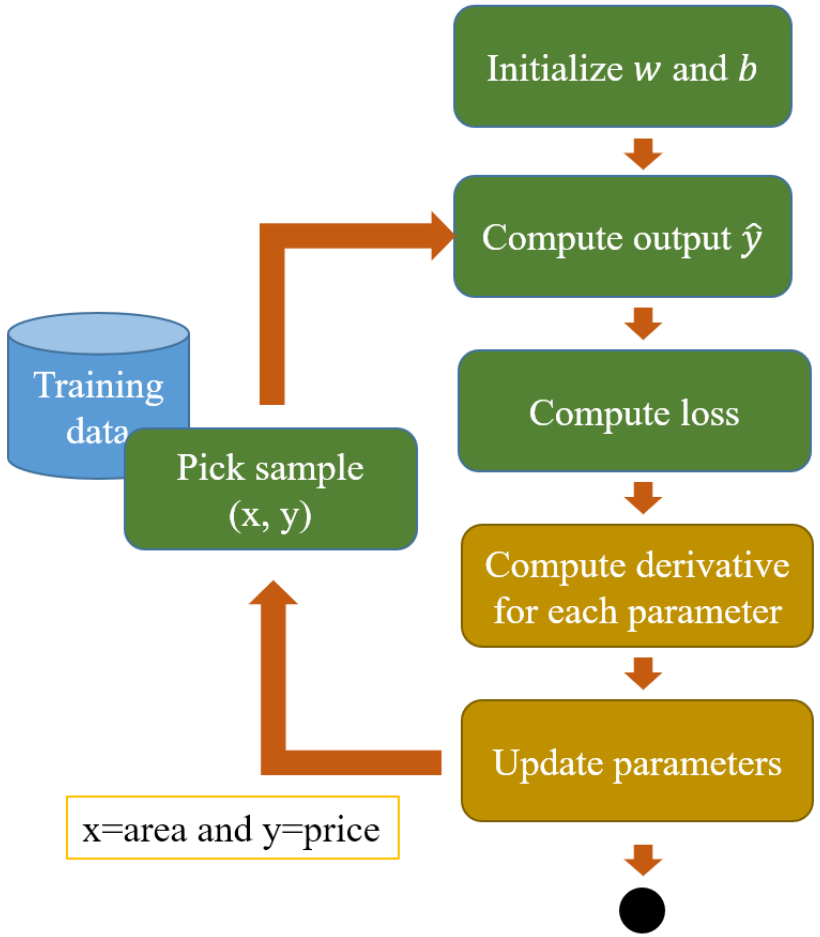
Gradient descent

SECTION 2

Linear Regression

SECTION 3

Vectorization



Linear Regression

Introduction

Feature | Label

area	price
6.7	9.1
4.6	5.9
3.5	4.6
5.5	6.7

House price data

Features | Label

TV	Radio	Newspaper	Sales
230.1	37.8	69.2	22.1
44.5	39.3	45.1	10.4
17.2	45.9	69.3	12
151.5	41.3	58.5	16.5
180.8	10.8	58.4	17.9

Advertising data

if area=6.0, price=?

if TV=55.0, Radio=34.0, and Newspaper=62.0, price=?

Features | Label

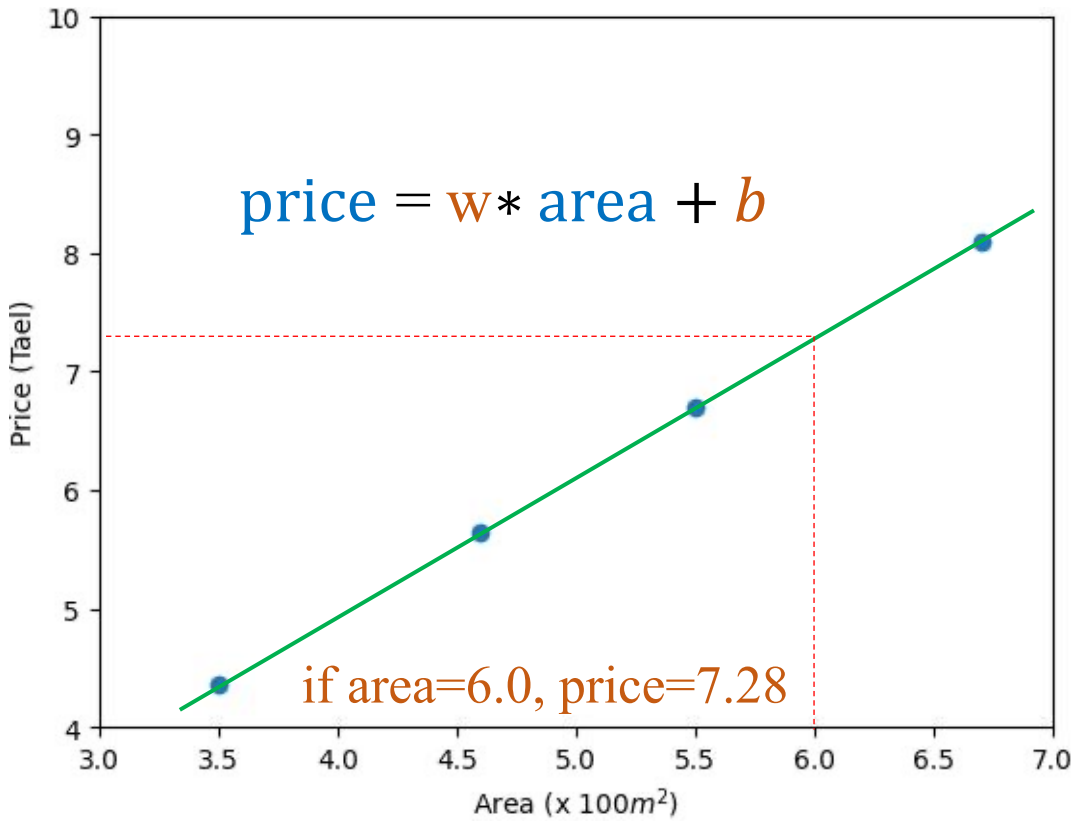
crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	black	lstat	medv
0.00632	18	2.31	0	0.538	6.575	65.2	4.09	1	296	15.3	396.9	4.98	24
0.02731	0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.9	9.14	21.6
0.03237	0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
0.06905	0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.9	5.33	36.2
0.08829	12.5	7.87	0	0.524	6.012	66.6	5.5605	5	311	15.2	395.6	12.43	22.9

Boston House Price Data

House Price Prediction

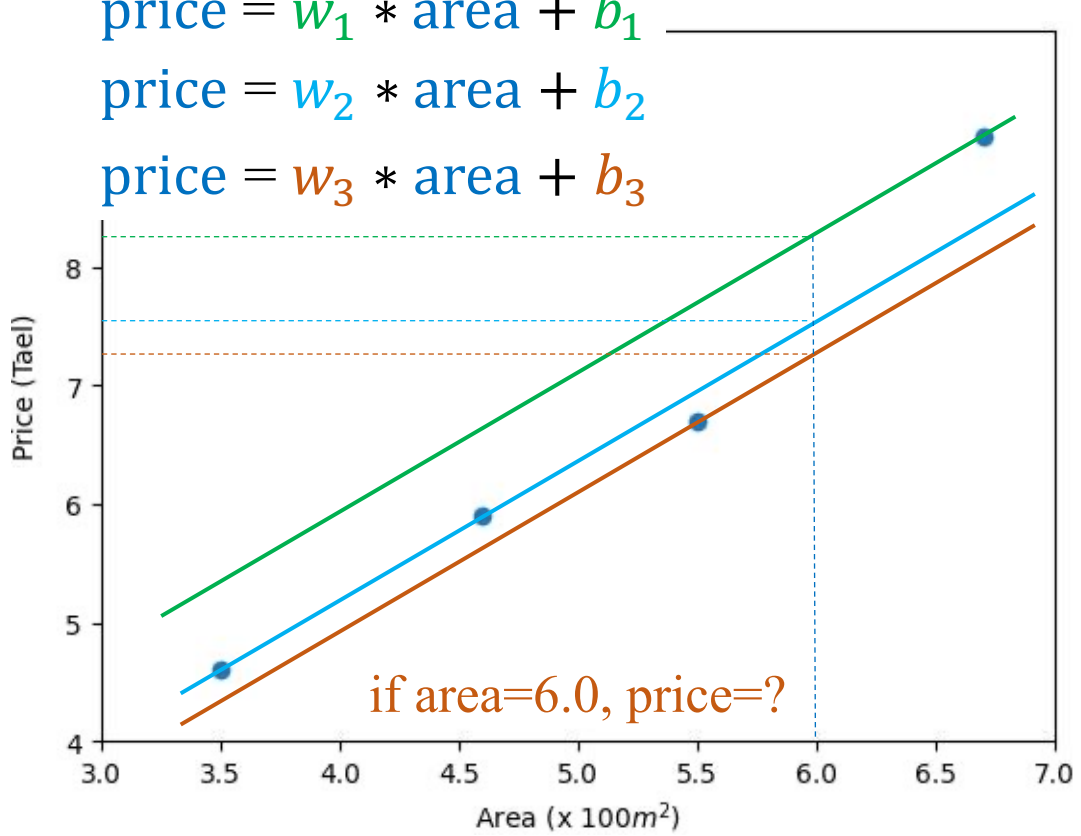
House price data

Feature	Label
area	price
6.7	8.1
4.6	5.6
3.5	4.3
5.5	6.7



House price data

Feature	Label
area	price
6.7	9.1
4.6	5.9
3.5	4.6
5.5	6.7

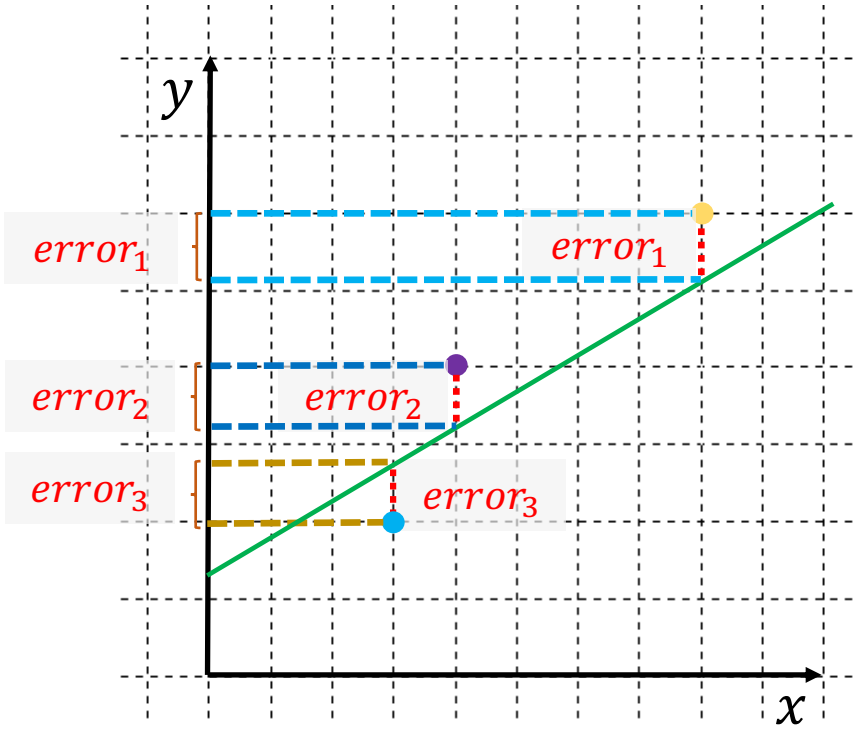


Linear Regression

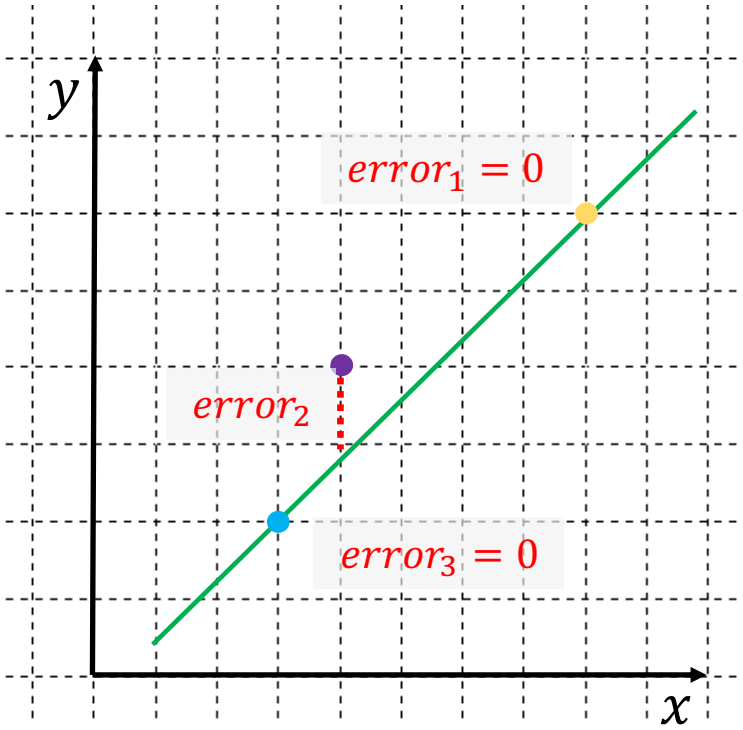
❖ Area-based house price prediction

Training data

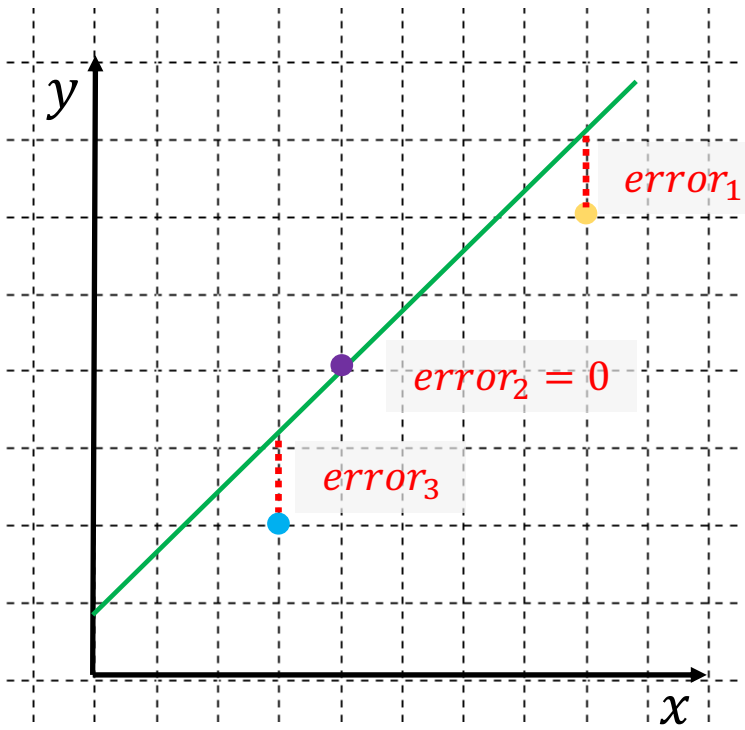
$$error_i = distance(\hat{y}_i, y_i)$$



$\hat{y} = w_1 x + b_1$



$\hat{y} = w_2 x + b_2$



$\hat{y} = w_3 x + b_3$

Find w and b whose model has the smallest error, where $error = \frac{1}{N}\sum_i error_i$ How?

Linear Regression

❖ Area-based house price prediction

weight bias

$$\text{predicted_price} = w * \text{area} + b$$

$$\text{error} = (\text{predicted_price} - \text{real_price})^2$$

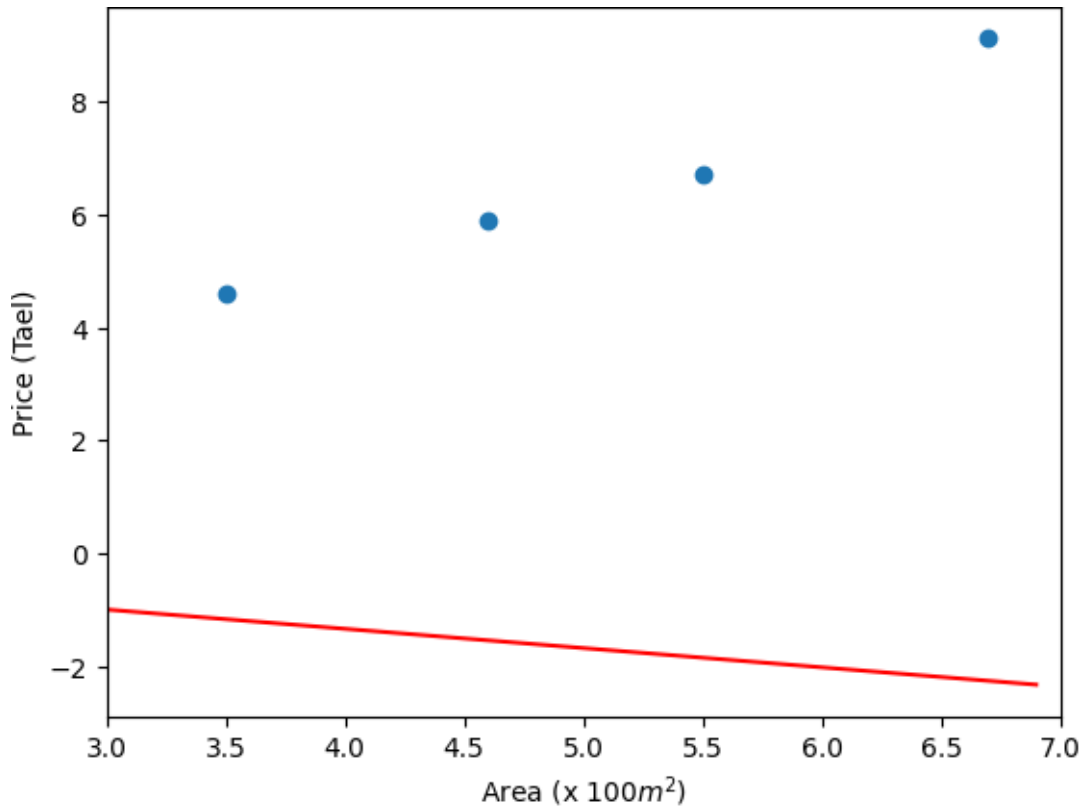
$$\hat{y}_i = wx_i + b$$

$$L(\hat{y}_i, y_i) = (\hat{y}_i - y_i)^2$$

$$w = -0.34$$

$$b = 0.04$$

area	price	predicted	error
6.7	9.1	-2.238	128.55
4.6	5.9	-1.524	55.11
3.5	4.6	-1.15	33.06
5.5	6.7	-1.83	72.76



Linear Regression

❖ Area-based house price prediction

$$\text{predicted_price} = w * \text{area} + b$$

$$\text{error} = (\text{predicted_price} - \text{real_price})^2$$

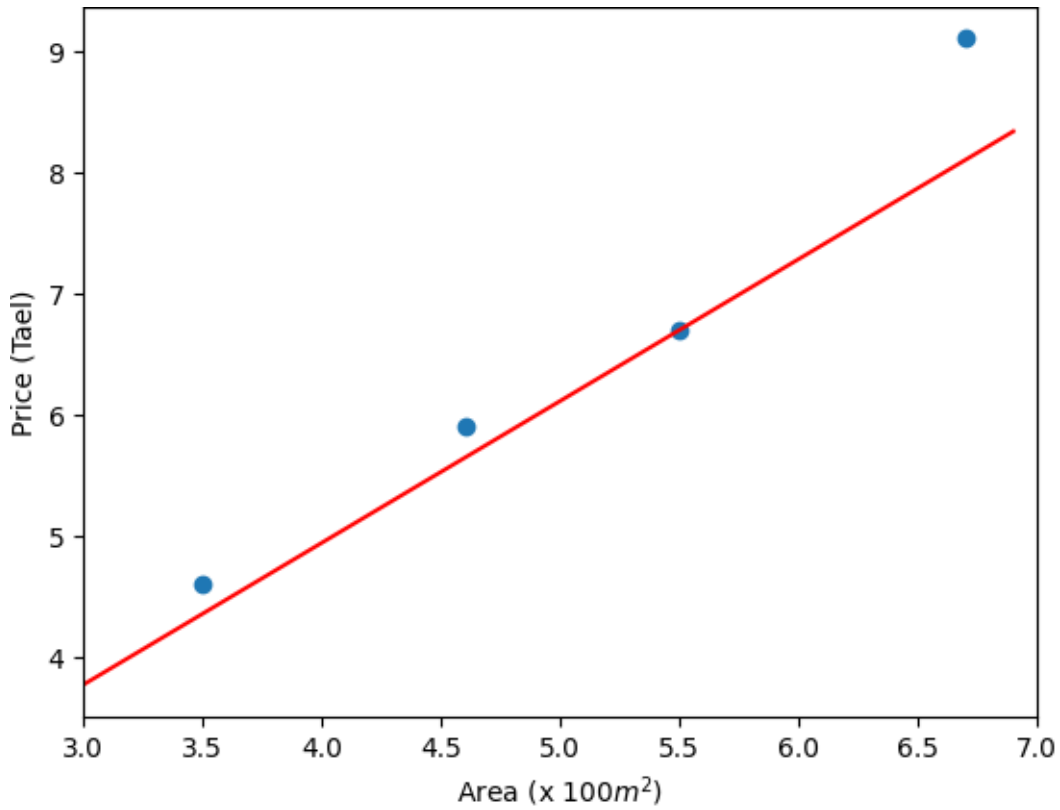
$$\hat{y}_i = wx_i + b$$

$$L(\hat{y}_i, y_i) = (\hat{y}_i - y_i)^2$$

area	price	predicted	error
6.7	9.1	8.099	1.002
4.6	5.9	5.642	0.066
3.5	4.6	4.355	0.06
5.5	6.7	6.695	0.00002

$w = 1.17$

$b = 0.26$



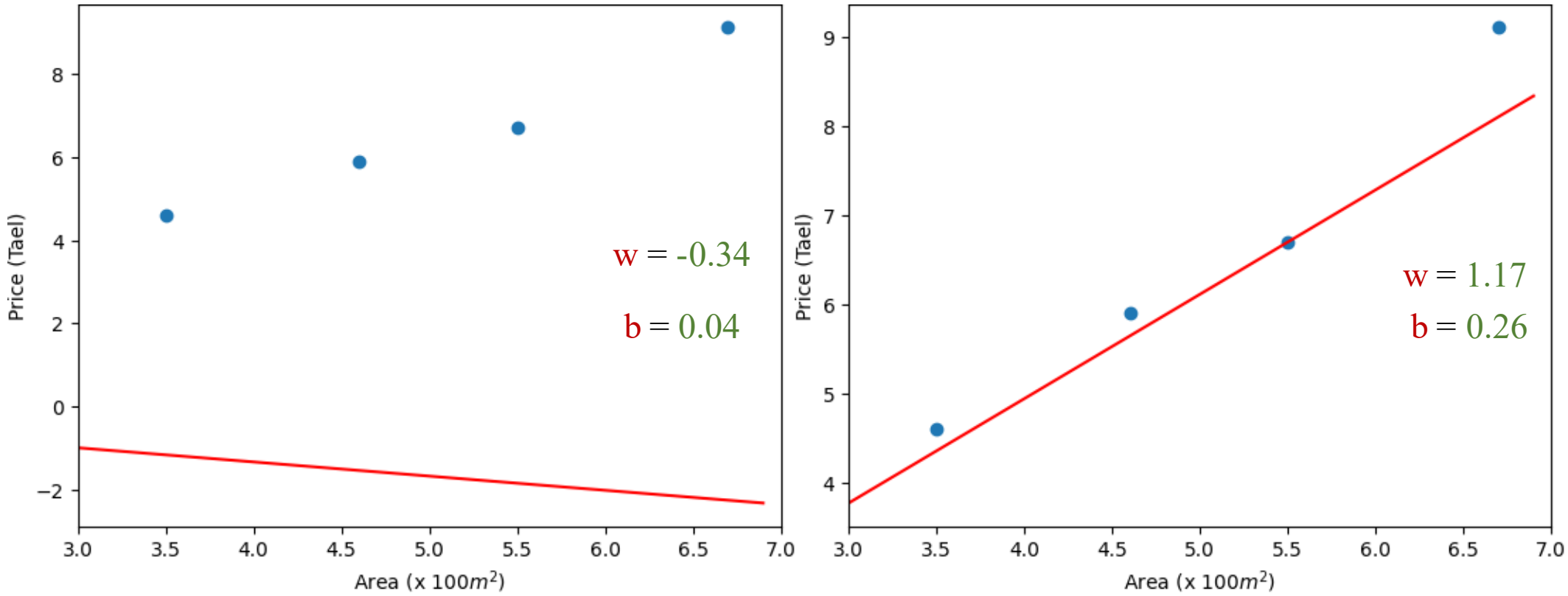
Linear Regression

❖ **Area-based house price prediction**

$$\hat{y}_i = wx_i + b$$

$$L(\hat{y}_i, y_i) = (\hat{y}_i - y_i)^2$$

How to change w and b so that $L(\hat{y}_i, y_i)$ reduces



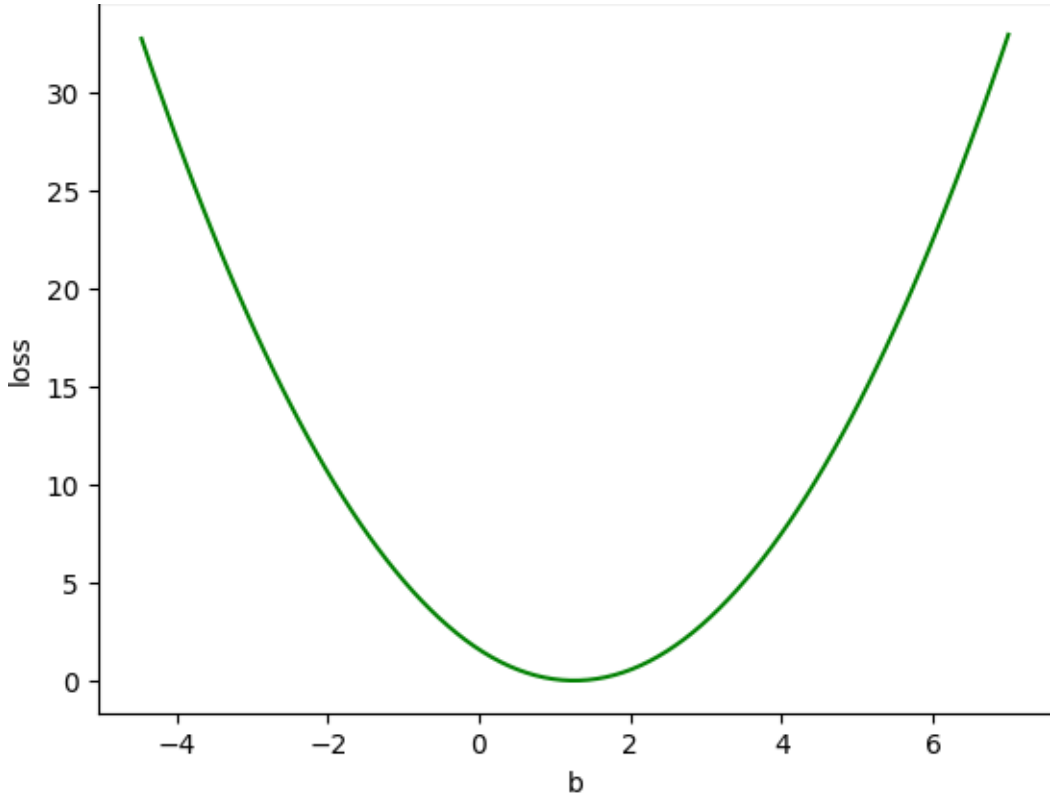
Linear Regression

$$\hat{y}_i = wx_i + b$$

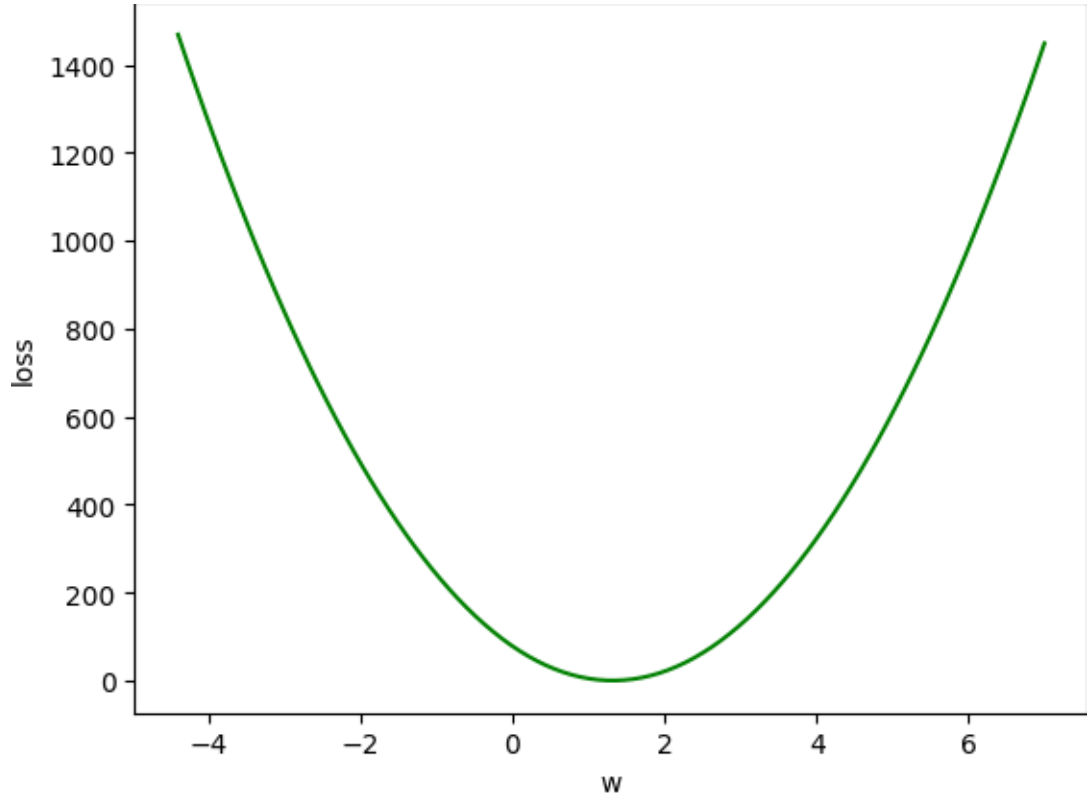
$$L(\hat{y}_i, y_i) = (\hat{y}_i - y_i)^2$$

How to change w and b so that $L(\hat{y}_i, y_i)$ reduces

❖ Understanding the loss function



Different b values with a fixed w value



Different w values with a fixed b value

Linear Regression

Linear equation

$$\hat{y} = wx + b$$

where $\hat{y}$ is a predicted value,

w and b are parameters

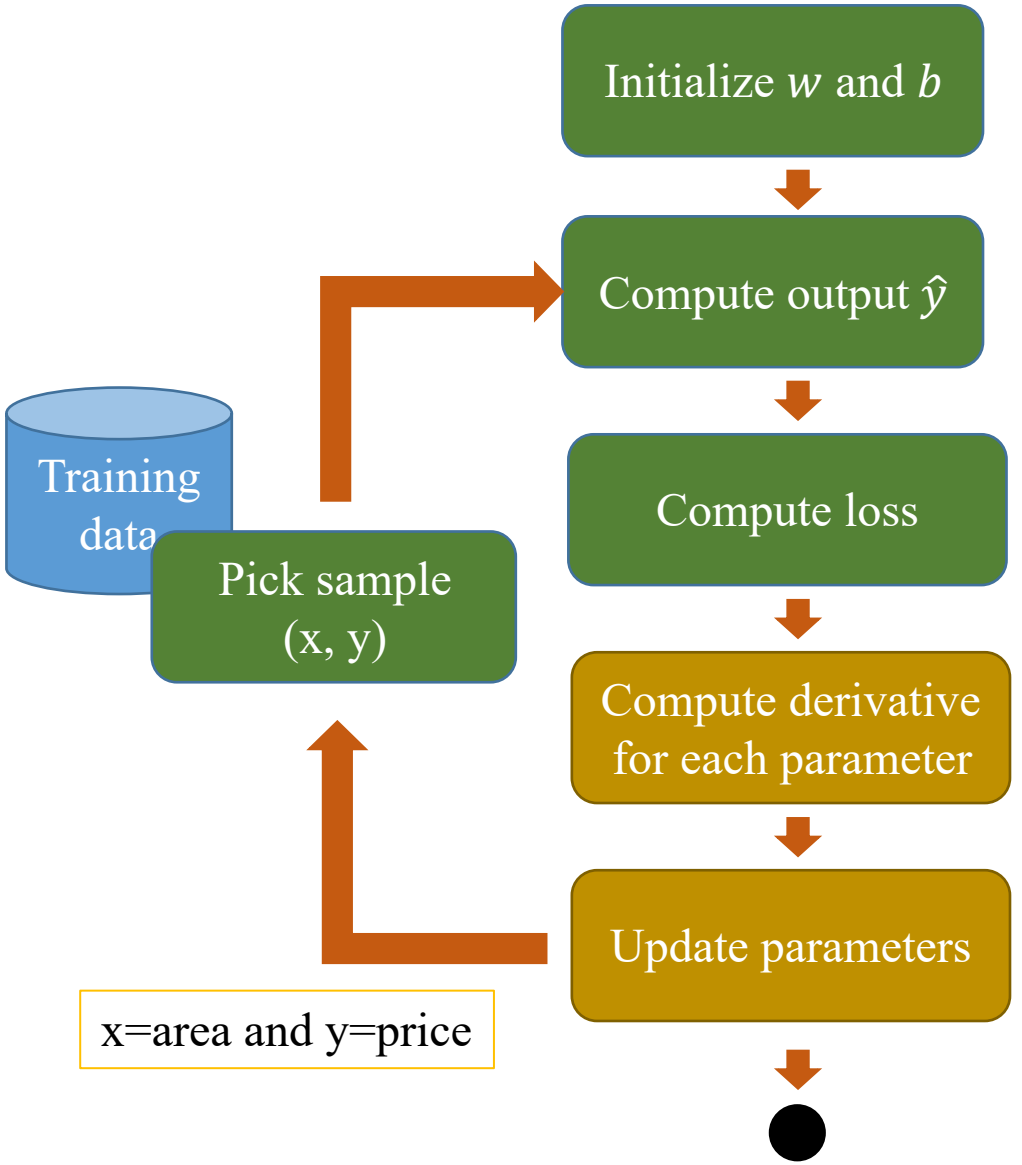
and x is an input feature

Error (loss) computation

Idea: compare predicted values $\hat{y}$ and label values y

Squared loss

$$L(\hat{y}, y) = (\hat{y} - y)^2$$



Linear Regression

Linear equation

$$\hat{y} = wx + b$$

where $\hat{y}$ is a predicted value,

w and b are parameters

and x is an input feature

Error (loss) computation

Idea: compare predicted values $\hat{y}$ and label values y

Squared loss

$$L(\hat{y}, y) = (\hat{y} - y)^2$$

Find better w and b

Use gradient descent to minimize the loss function

Compute derivate for each parameter

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial w} = 2x(\hat{y} - y)$$

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial b} = 2(\hat{y} - y)$$

Update parameters

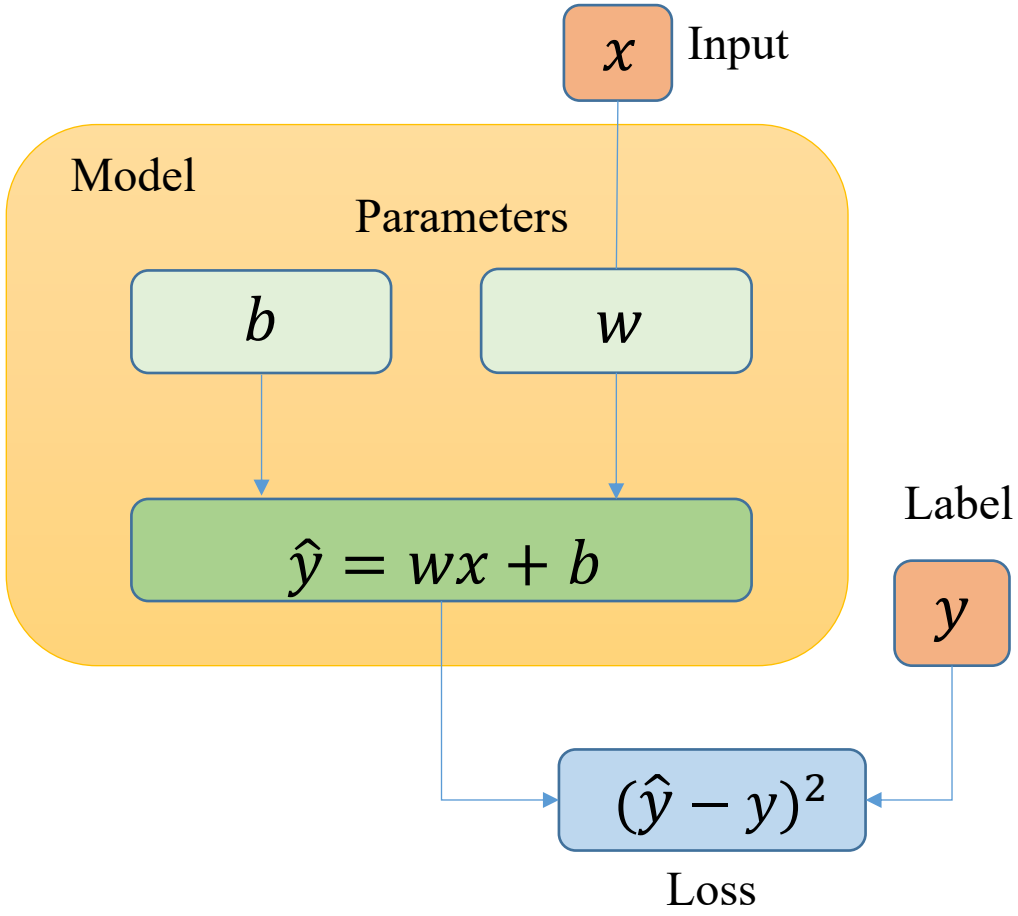
$$w = w - \eta \frac{\partial L}{\partial w} \qquad b = b - \eta \frac{\partial L}{\partial b}$$

η is learning rate

Linear Regression

❖ Simple example

Diagram



Cheat sheet

Compute the output $\hat{y}$

$$\hat{y} = wx + b$$

Compute the loss

$$L = (\hat{y} - y)^2$$

Compute derivative

$$\frac{\partial L}{\partial w} = 2x(\hat{y} - y)$$

$$\frac{\partial L}{\partial b} = 2(\hat{y} - y)$$

Update parameters

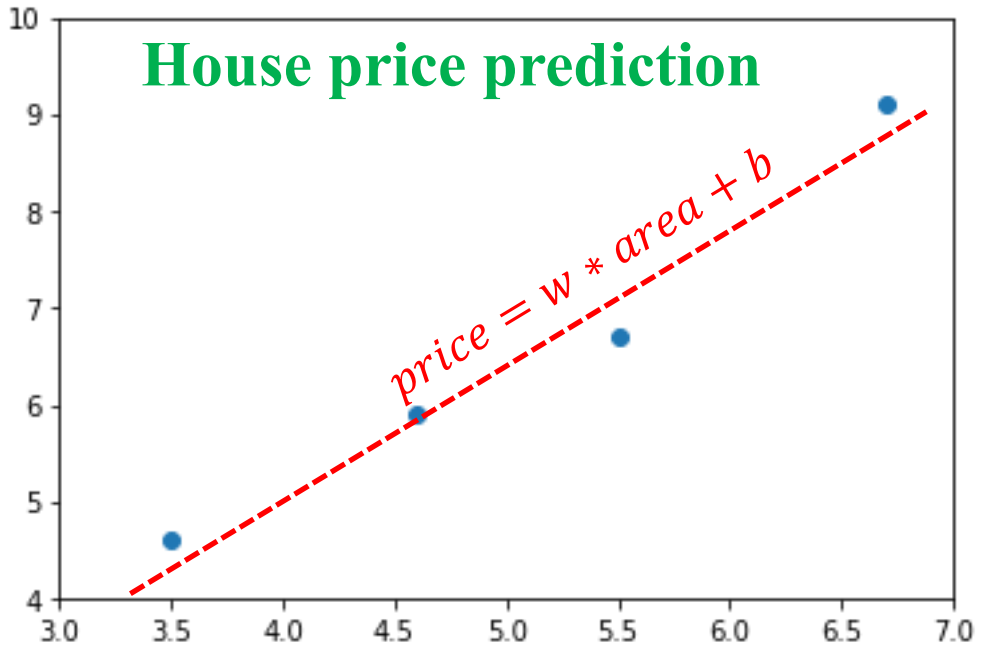
$$w = w - \eta \frac{\partial L}{\partial w}$$

$$b = b - \eta \frac{\partial L}{\partial b}$$

Linear Regression

Given sample data

Feature	Label
area	price
6.7	9.1
4.6	5.9
3.5	4.6
5.5	6.7



Initialize b=0.04 and w=-0.34

1

Input $x = 6.7$

Model

Parameters

$b = 0.04$

$w = -0.34$

$\hat{y} = xw + b = -2.238$

Label

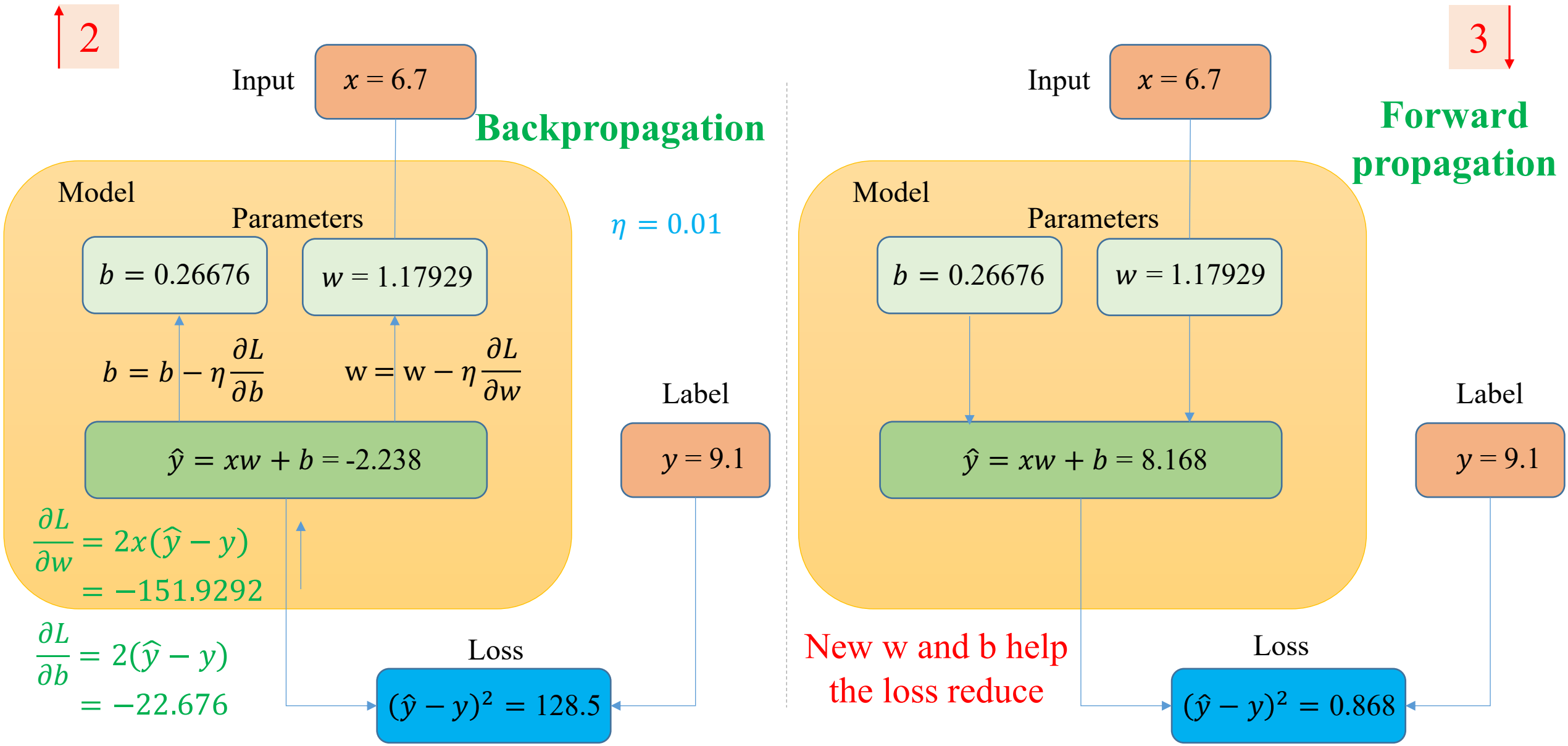
$y = 9.1$

Loss

$(\hat{y} - y)^2 = 128.5$

Forward propagation

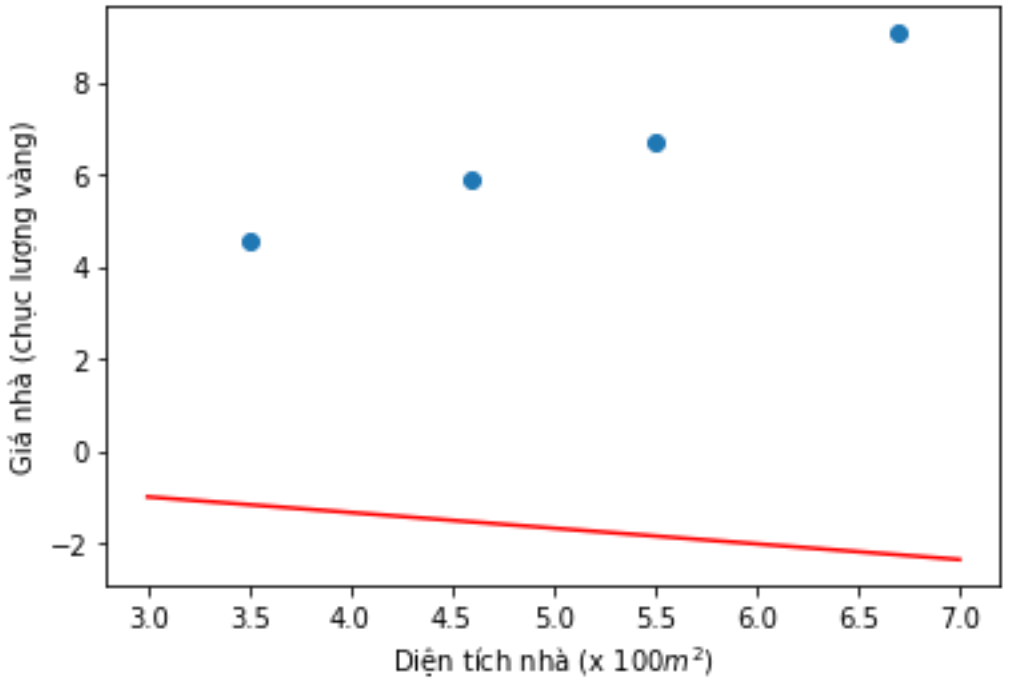
Linear Regression



Linear Regression

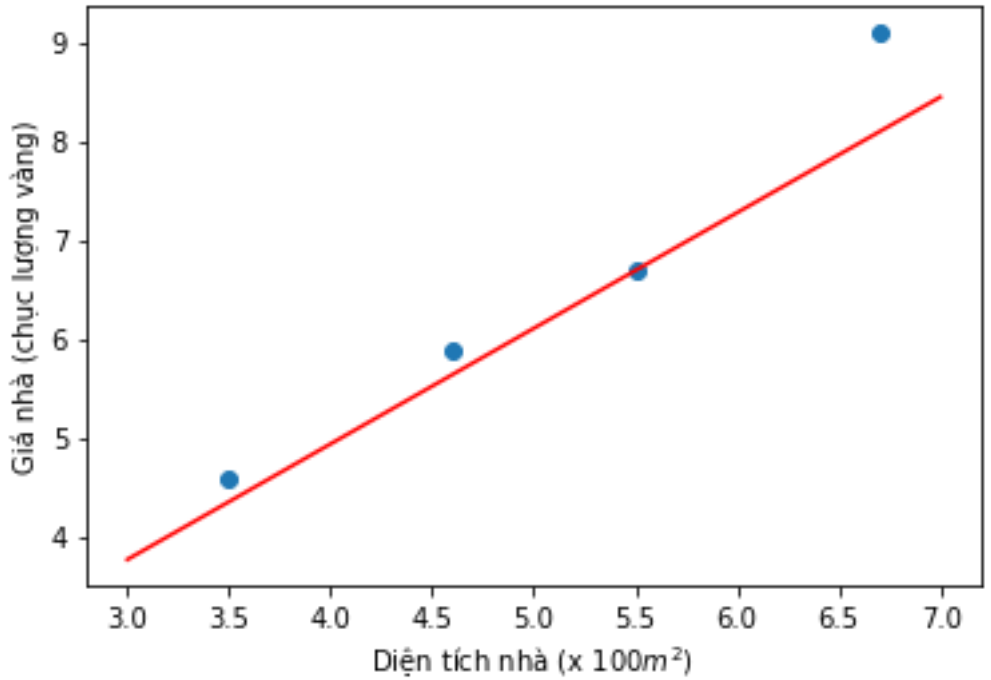
❖ Simple example

Model prediction before and after the first update



w = -0.34 b = 0.04 L = 128.55

Before updating

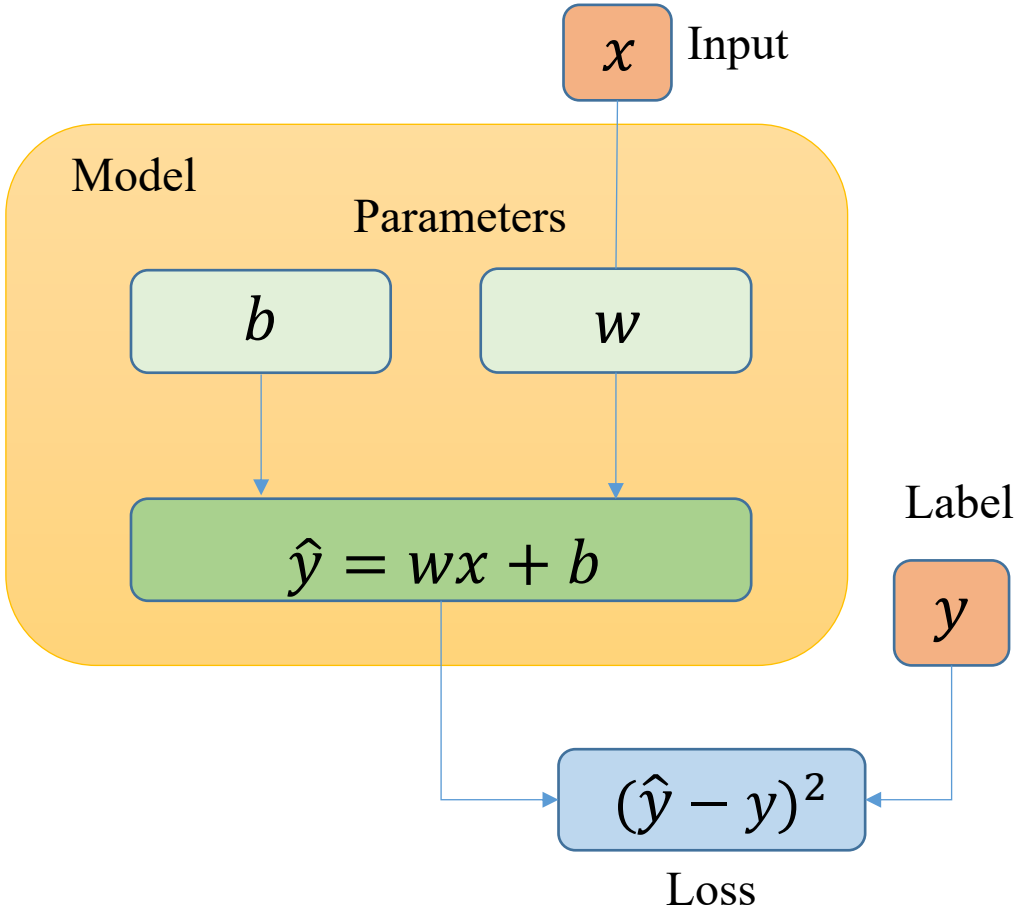


w = 1.179292 b = 0.26676 L = 0.868

After updating

Linear Regression

❖ Summary (simple version)



1) Pick a sample (x, y) from training data

2) Compute the output $\hat{y}$

$$\hat{y} = wx + b$$

3) Compute loss

$$L = (\hat{y} - y)^2$$

4) Compute derivative

$$\frac{\partial L}{\partial w} = 2x(\hat{y} - y) \qquad \frac{\partial L}{\partial b} = 2(\hat{y} - y)$$

5) Update parameters

$$w = w - \eta \frac{\partial L}{\partial w} \qquad b = b - \eta \frac{\partial L}{\partial b}$$

η is learning rate

Implementation

❖ One-sample training

Feature	Label
area	price
6.7	9.1
4.6	5.9
3.5	4.6
5.5	6.7

column index=0 column index=1

```
# data preparation
import numpy as np
import matplotlib.pyplot as plt

data = np.genfromtxt('data.csv',
                     delimiter=',')

x_data = data[:,0]
y_data = data[:,1]
N = data.shape[0]

print(f'areas: {x_data}')
print(f'prices: {y_data}')
print(f'data_size: {N}')
```

✓ 0.2s

```
areas: [6.7 4.6 3.5 5.5]
prices: [9.1 5.9 4.6 6.7]
data_size: 4
```

Implementation

❖ House price prediction

❖ One-sample training

Demo

1) Pick a sample (x, y) from training data

2) Compute the output $\hat{y}$

$$\hat{y} = wx + b$$

3) Compute loss

$$L = (\hat{y} - y)^2$$

4) Compute derivative

$$\frac{\partial L}{\partial w} = 2x(\hat{y} - y) \qquad \frac{\partial L}{\partial b} = 2(\hat{y} - y)$$

5) Update parameters

$$w = w - \eta \frac{\partial L}{\partial w} \qquad b = b - \eta \frac{\partial L}{\partial b}$$

```python
# forward
def predict(x, w, b):
    return x*w + b

# compute gradient
def gradient(y_hat, y, x):
    dw = 2*x*(y_hat-y)
    db = 2*(y_hat-y)

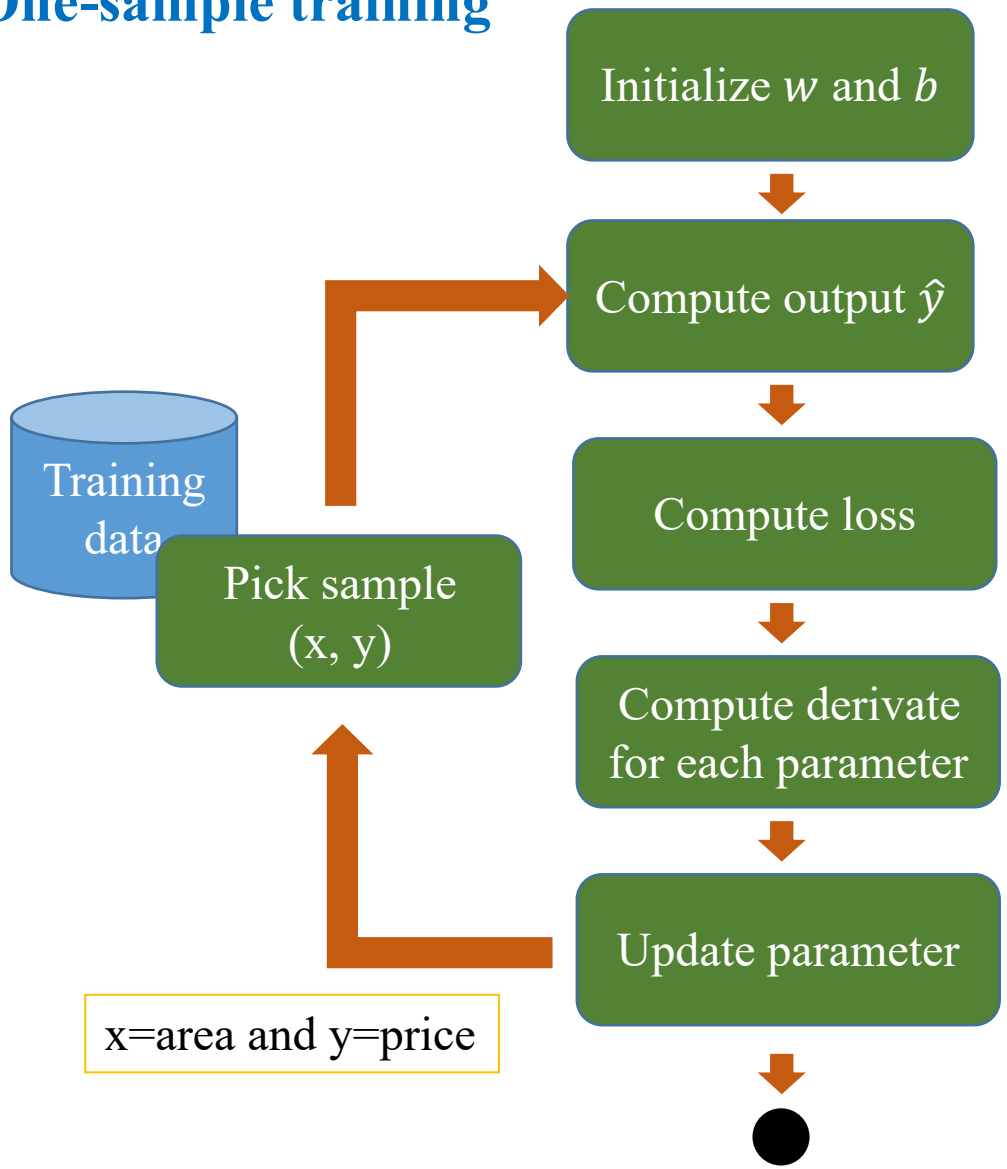
    return (dw, db)

# update weights
def update_weight(w, b, lr, dw, db):
    w_new = w - lr*dw
    b_new = b - lr*db

    return (w_new, b_new)
```

Implementation

❖ **One-sample training**




```python
# init weights
b = 0.04
w = -0.34
lr = 0.01

# how long
epoch_max = 10

for epoch in range(epoch_max):
    for i in range(data_size):
        # get a sample
        # ...

        # predict y_hat
        # ...

        # compute loss
        # ...

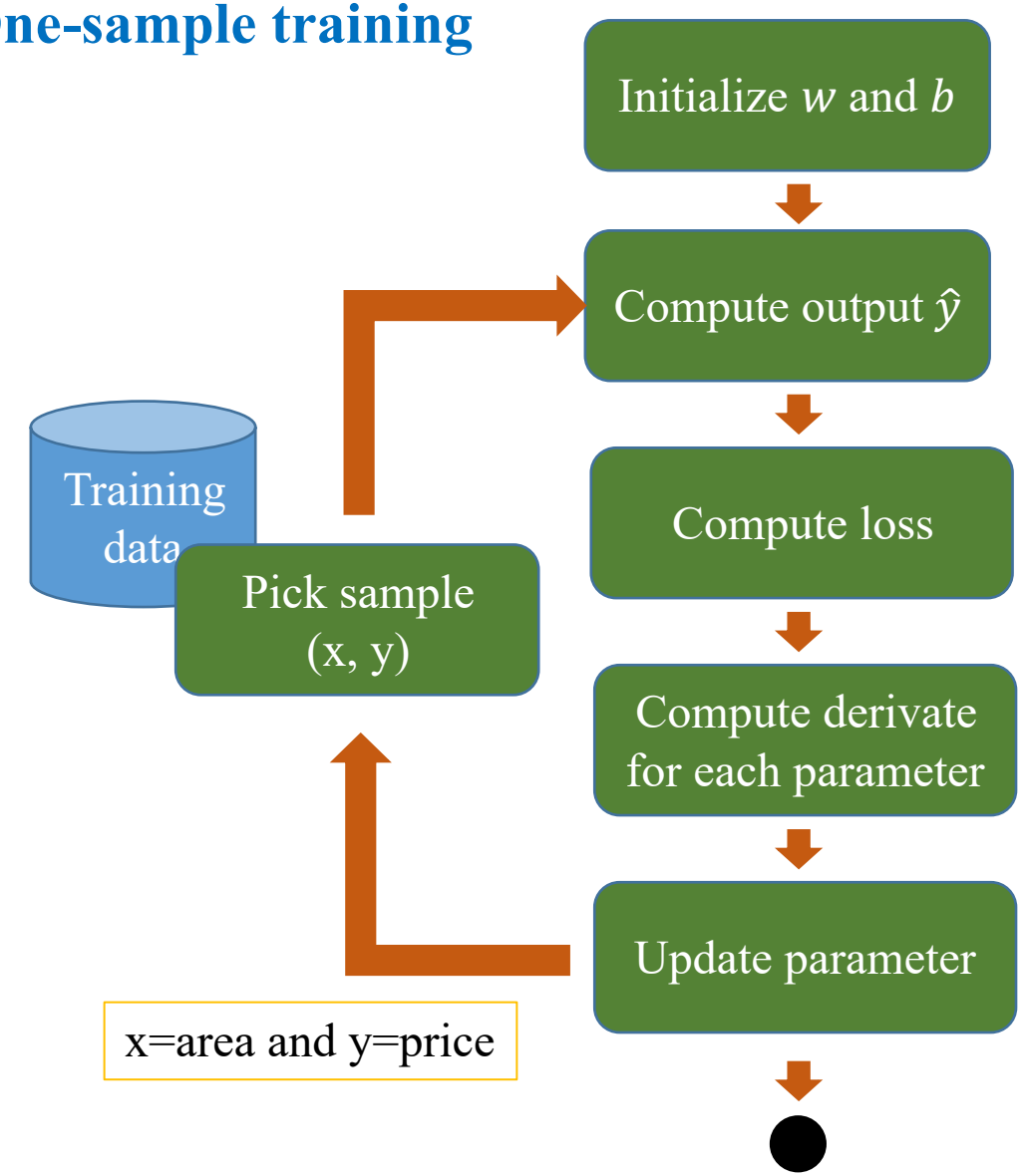
        # compute gradient
        # ...

        # update weights
        # ...

```

Implementation

❖ One-sample training




```python
# init weights
b = 0.04
w = -0.34
lr = 0.01

# how long
epoch_max = 10
data_size = 4

for epoch in range(epoch_max):
    for i in range(data_size):
        # get a sample
        x = areas[i]
        y = prices[i]

        # predict y_hat
        y_hat = predict(x, w, b)

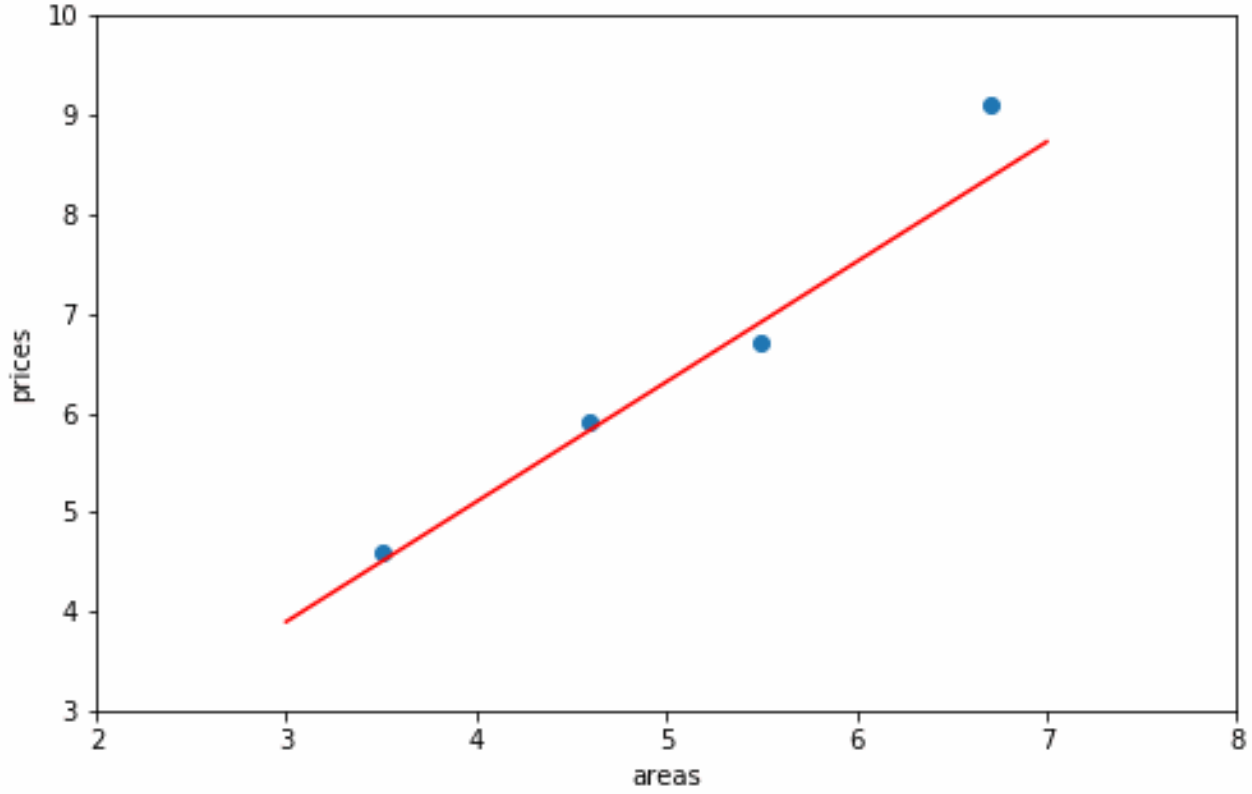
        # compute loss
        loss = (y_hat-y)*(y_hat-y)

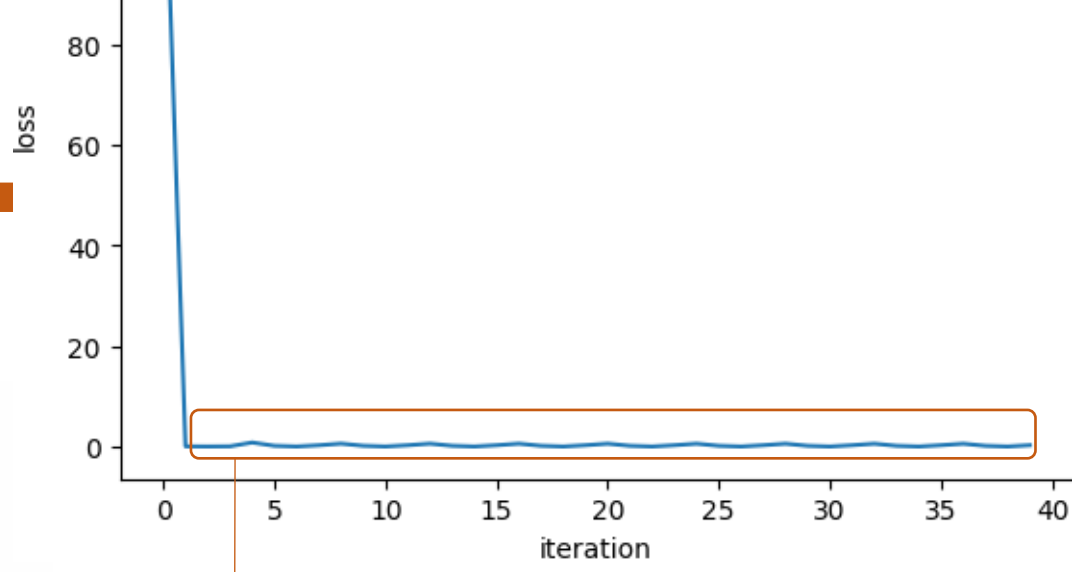
        # compute gradient
        (dw, db) = gradient(y_hat, y, x)

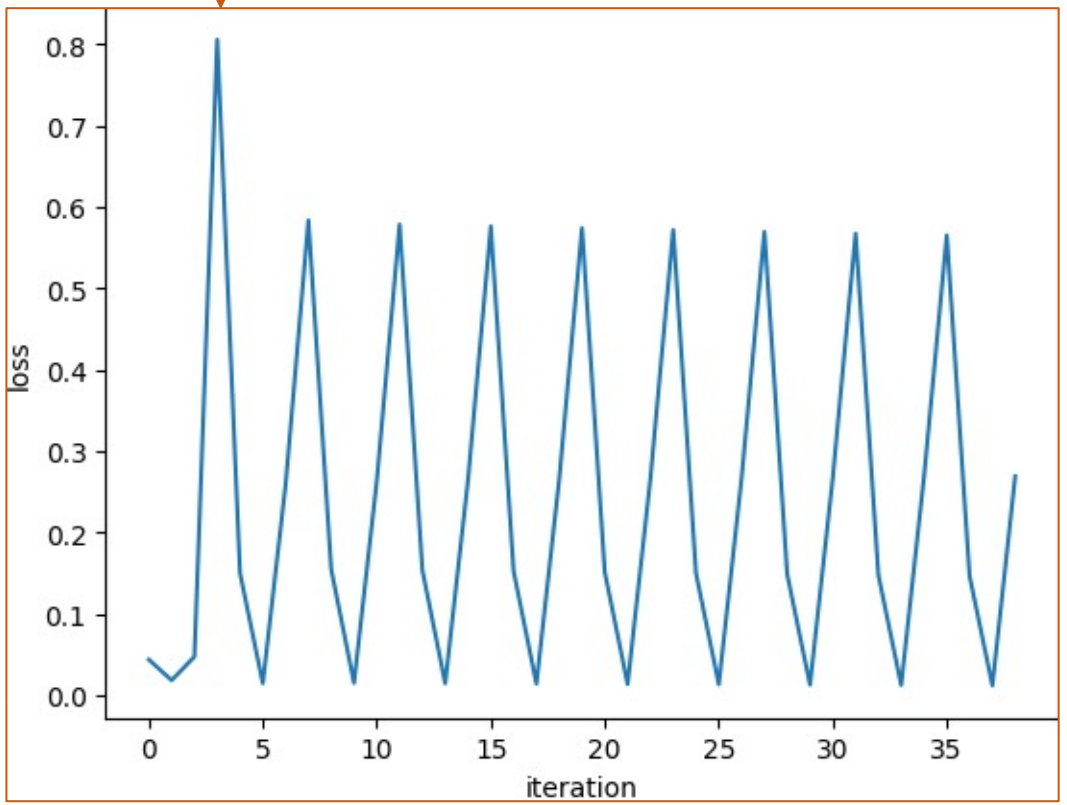
        # update weights
        (w, b) = update_weight(w, b, lr, dw, db)
```

Implementation

- House price prediction
  - One-sample training







Model training

Linear Regression

❖ General formula

Feature	Label
area	price
6.7	9.1
4.6	5.9
3.5	4.6
5.5	6.7

House price data

Model: $\hat{y} = w_1 x_1 + b$

$\widehat{\text{price}} = w_1 * area + b$

Features			Label
TV	Radio	Newspaper	Sales
230.1	37.8	69.2	22.1
44.5	39.3	45.1	10.4
17.2	45.9	69.3	12
151.5	41.3	58.5	16.5
180.8	10.8	58.4	17.9

Advertising data

Model: $\hat{y} = w_1 x_1 + w_2 x_2 + w_3 x_3 + b$

$\widehat{\text{Sales}} = w_1 * TV + w_2 * Radio + w_3 * Newspaper + b$

Linear Regression

1) Pick a sample (x_1, x_2, x_3, y) from training data

2) Compute the output $\hat{y}$

$$\hat{y} = w_1 * TV + w_2 * R + w_3 * N + b$$

$$\hat{y} = w_1 * x_1 + w_2 * x_2 + w_3 * x_3 + b$$

3) Compute loss

$$L = (\hat{y} - y)^2$$

4) Compute derivative

$$\frac{\partial L}{\partial w_1} = 2x_1(\hat{y} - y) \qquad \frac{\partial L}{\partial w_3} = 2x_3(\hat{y} - y)$$

$$\frac{\partial L}{\partial w_2} = 2x_2(\hat{y} - y) \qquad \frac{\partial L}{\partial b} = 2(\hat{y} - y)$$

5) Update parameters

$$w_1 = w_1 - \eta \frac{\partial L}{\partial w_1} \qquad w_3 = w_3 - \eta \frac{\partial L}{\partial w_3}$$

$$w_2 = w_2 - \eta \frac{\partial L}{\partial w_2} \qquad b = b - \eta \frac{\partial L}{\partial b}$$

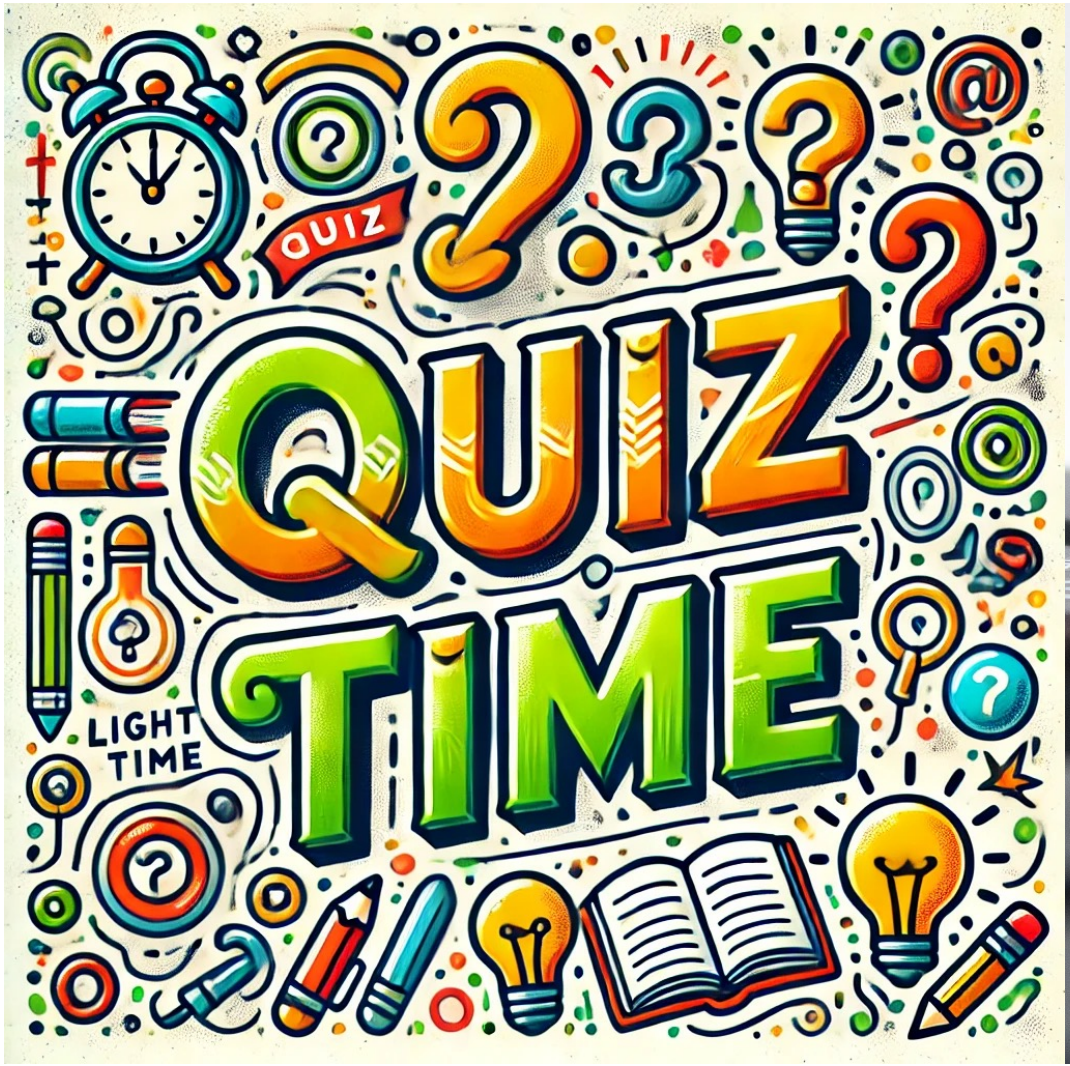
Features			Label
TV	Radio	Newspaper	Sales
230.1	37.8	69.2	22.1
44.5	39.3	45.1	10.4
17.2	45.9	69.3	12
151.5	41.3	58.5	16.5
180.8	10.8	58.4	17.9

Advertising data

Model

$$\text{Sale} = w_1 * TV + w_2 * Radio + w_3 * Newspaper + b$$

$$\hat{y} = w_1 x_1 + w_2 x_2 + w_3 x_3 + b$$


QUIZ
QUIZ
TIME
LIGHT
TIME


BREAK
TIME!!!

Outline

SECTION 1

Gradient descent

SECTION 2

Linear Regression

SECTION 3

Vectorization

$$\begin{cases} b = b - \eta \dfrac{\partial L}{\partial b} \\ w = w - \eta \dfrac{\partial L}{\partial w} \end{cases}$$

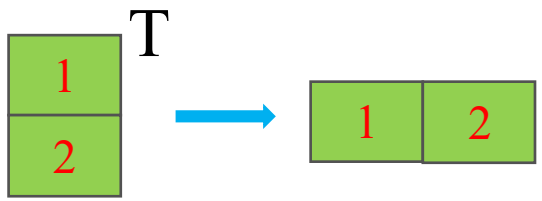
$$\boldsymbol{\theta} \quad \boldsymbol{\theta} \quad \nabla_{\boldsymbol{\theta}} \mathrm{L}$$

$$\rightarrow \quad \boldsymbol{\theta} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \mathrm{L}$$

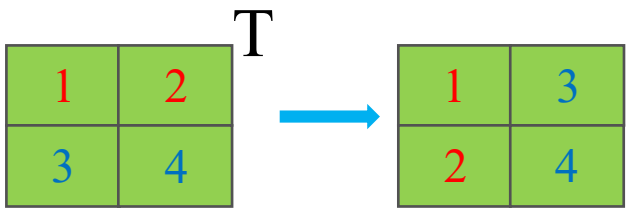
Linear Regression (1-samples)

Transpose

$$\vec{v} = \begin{bmatrix} v_1 \\ \dots \\ v_n \end{bmatrix} \qquad \vec{v}^T = [v_1 \quad \dots v_n]$$



$$\mathrm{A} = \begin{bmatrix} a_{11} & \dots & a_{1n} \\ \dots & \dots & \dots \\ a_{m1} & \dots & a_{mn} \end{bmatrix} \qquad \mathrm{A}^T = \begin{bmatrix} a_{11} & \dots & a_{m1} \\ \dots & \dots & \dots \\ a_{1n} & \dots & a_{mn} \end{bmatrix}$$




```python
import numpy as np

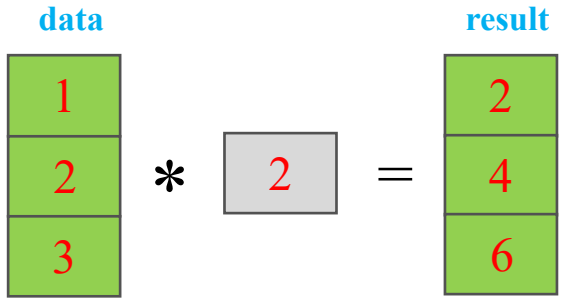
# create data
data = np.array([1,2,3])
factor = 2

# broadcasting
result_multiplication = data*factor
```

```
[1 2 3]
[2 4 6]
```

Multiply with a number

$$\alpha\vec{u} = \alpha \begin{bmatrix} u_1 \\ \dots \\ u_n \end{bmatrix} = \begin{bmatrix} \alpha u_1 \\ \dots \\ \alpha u_n \end{bmatrix}$$



Linear Regression (1-samples)

Feature	Label
area	price
6.7	9.1
4.6	5.9
3.5	4.6
5.5	6.7
x	y

Traditional

1) Pick a sample (x, y) from training data

2) Compute the output $\hat{y}$

$$\hat{y} = wx + b$$

3) Compute loss

$$L = (\hat{y} - y)^2$$

4) Compute derivative

$$\frac{\partial L}{\partial w} = 2x(\hat{y} - y) \qquad \frac{\partial L}{\partial b} = 2(\hat{y} - y)$$

5) Update parameters

$$w = w - \eta \frac{\partial L}{\partial w} \qquad b = b - \eta \frac{\partial L}{\partial b}$$

η is learning rate

$$\hat{y} = wx + b \qquad \boldsymbol{x} = \begin{bmatrix} 1 \\ x \end{bmatrix} \qquad \boldsymbol{\theta} = \begin{bmatrix} b \\ w \end{bmatrix}$$

$$\boldsymbol{\theta} = \begin{bmatrix} b \\ w \end{bmatrix} \rightarrow \boldsymbol{\theta}^T = [b \quad w]$$

$$\hat{y} = wx + b1 = [b \quad w] \begin{bmatrix} 1 \\ x \end{bmatrix} = \boldsymbol{\theta}^T \boldsymbol{x}$$

dot product

Linear Regression (1-samples)

1) Pick a sample (x, y) from training data

2) Compute the output $\hat{y}$

$$\hat{y} = wx + b$$

3) Compute loss

$$L = (\hat{y} - y)^2$$

4) Compute derivative

$$\frac{\partial L}{\partial w} = 2x(\hat{y} - y) \qquad \frac{\partial L}{\partial b} = 2(\hat{y} - y)$$

5) Update parameters

$$w = w - \eta \frac{\partial L}{\partial w} \qquad b = b - \eta \frac{\partial L}{\partial b}$$

η is learning rate

Traditional

$$\hat{y} = wx + b \qquad \boldsymbol{x} = \begin{bmatrix} 1 \\ x \end{bmatrix} \qquad \boldsymbol{\theta} = \begin{bmatrix} b \\ w \end{bmatrix}$$

$$\hat{y} = \boldsymbol{\theta}^T \boldsymbol{x}$$

$$L(\hat{y}, y) = (\hat{y} - y)^2$$

numbers

What will we do?

Linear Regression (1-samples)

Traditional

1) Pick a sample (x, y) from training data

2) Compute the output $\hat{y}$

$$\hat{y} = wx + b$$

3) Compute loss

$$L = (\hat{y} - y)^2$$

4) Compute derivative

$$\frac{\partial L}{\partial w} = 2x(\hat{y} - y) \qquad \frac{\partial L}{\partial b} = 2(\hat{y} - y)$$

5) Update parameters

$$w = w - \eta \frac{\partial L}{\partial w} \qquad b = b - \eta \frac{\partial L}{\partial b}$$

η is learning rate

$$\hat{y} = wx + b \qquad \boldsymbol{x} = \begin{bmatrix} 1 \\ x \end{bmatrix} \qquad \boldsymbol{\theta} = \begin{bmatrix} b \\ w \end{bmatrix}$$

$$\begin{cases} \frac{\partial L}{\partial b} = 2(\hat{y} - y) = 2 \times (\hat{y} - y) \times 1 \\ \frac{\partial L}{\partial w} = 2x(\hat{y} - y) = 2 \times (\hat{y} - y) \times x \end{cases}$$

$$\begin{bmatrix} 2 \times (\hat{y} - y) \times 1 \\ 2 \times (\hat{y} - y) \times x \end{bmatrix} = 2(\hat{y} - y) \begin{bmatrix} 1 \\ x \end{bmatrix} = 2(\hat{y} - y)\boldsymbol{x} = \begin{bmatrix} \frac{\partial L}{\partial b} \\ \frac{\partial L}{\partial w} \end{bmatrix} = \nabla_{\boldsymbol{\theta}} \mathrm{L} \quad \rightarrow \quad \nabla_{\boldsymbol{\theta}} \mathrm{L} = 2\boldsymbol{x}(\hat{y} - y)$$

common factor

quiz

Linear Regression (1-samples)

$$\hat{y} = wx + b$$

$$\boldsymbol{x} = \begin{bmatrix} 1 \\ x \end{bmatrix}$$

$$\boldsymbol{\theta} = \begin{bmatrix} b \\ w \end{bmatrix}$$

$$\nabla_{\boldsymbol{\theta}} \mathrm{L} = \begin{bmatrix} \frac{\partial L}{\partial b} \\ \frac{\partial L}{\partial w} \end{bmatrix}$$

Traditional

1) Pick a sample (x, y) from training data

2) Compute the output $\hat{y}$

$$\hat{y} = wx + b$$

3) Compute loss

$$L = (\hat{y} - y)^2$$

4) Compute derivative

$$\frac{\partial L}{\partial w} = 2x(\hat{y} - y) \qquad \frac{\partial L}{\partial b} = 2(\hat{y} - y)$$

5) Update parameters

$$w = w - \eta \frac{\partial L}{\partial w} \qquad b = b - \eta \frac{\partial L}{\partial b}$$

$$\begin{cases} b = b - \eta \dfrac{\partial L}{\partial b} \\ w = w - \eta \dfrac{\partial L}{\partial w} \end{cases}$$

$$\boldsymbol{\theta} \quad \boldsymbol{\theta} \quad \nabla_{\boldsymbol{\theta}} \mathrm{L}$$

$$\rightarrow \quad \boldsymbol{\theta} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \mathrm{L}$$

Linear Regression (1-samples)

Traditional

1) Pick a sample (x, y) from training data

2) Compute the output $\hat{y}$

$$\hat{y} = wx + b$$

3) Compute loss

$$L = (\hat{y} - y)^2$$

4) Compute derivative

$$\frac{\partial L}{\partial w} = 2x(\hat{y} - y) \qquad \frac{\partial L}{\partial b} = 2(\hat{y} - y)$$

5) Update parameters

$$w = w - \eta \frac{\partial L}{\partial w} \qquad b = b - \eta \frac{\partial L}{\partial b}$$

η is learning rate

Vectorized

1) Pick a sample (x, y) from training data

2) Compute output $\hat{y}$

$$\hat{y} = \boldsymbol{\theta}^T \boldsymbol{x} = \boldsymbol{x}^T \boldsymbol{\theta}$$

3) Compute loss

$$L = (\hat{y} - y)^2$$

4) Compute derivative

$$\nabla_{\boldsymbol{\theta}} \mathrm{L} = 2\boldsymbol{x}(\hat{y} - y)$$

5) Update parameters

$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \mathrm{L}$$

η is learning rate

$\hat{y} = wx + b$

$\boldsymbol{x} = \begin{bmatrix} 1 \\ x \end{bmatrix} \quad \boldsymbol{\theta} = \begin{bmatrix} b \\ w \end{bmatrix}$

Feature	Label
area	price
6.7	9.1
4.6	5.9
3.5	4.6
5.5	6.7
x	y

1) Pick a sample $(\mathbf{x}, y)$ from training data

2) Compute output $\hat{y}$

$$\hat{y} = \boldsymbol{\theta}^T \boldsymbol{x} = \boldsymbol{x}^T \boldsymbol{\theta}$$

3) Compute loss

$$L = (\hat{y} - y)^2$$

4) Compute derivative

$$\nabla_{\boldsymbol{\theta}} \mathrm{L} = 2\boldsymbol{x}(\hat{y} - y)$$

5) Update parameters

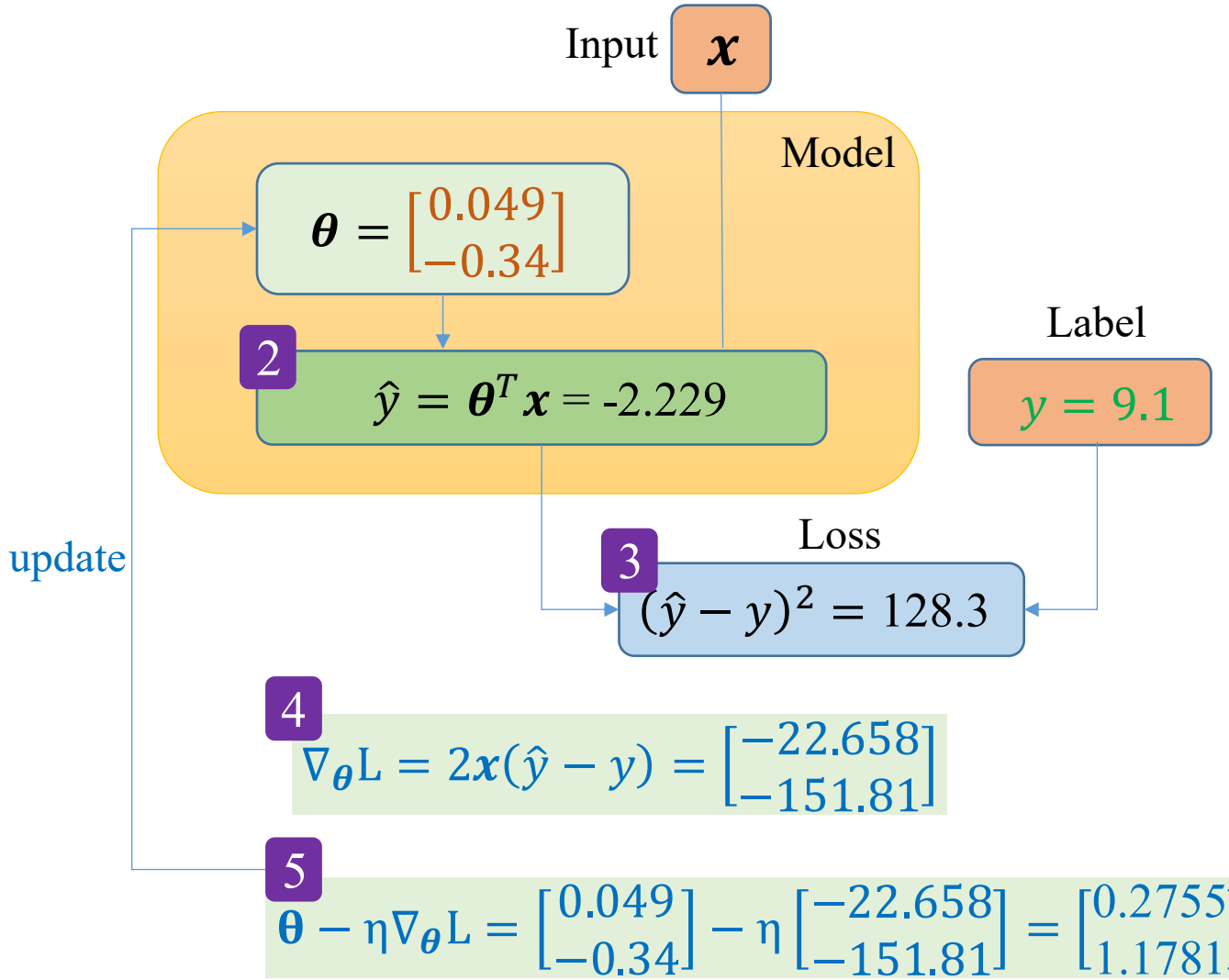
$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \mathrm{L}$$

η is learning rate

1 $\boldsymbol{x} = \begin{bmatrix} 1 \\ x \end{bmatrix} = \begin{bmatrix} 1 \\ 6.7 \end{bmatrix}$

Given $\boldsymbol{\theta} = \begin{bmatrix} b \\ w \end{bmatrix} = \begin{bmatrix} 0.049 \\ -0.34 \end{bmatrix}$

$\eta = 0.01$



Linear Regression (1-samples)

❖ Implementation (vectorization using numpy)

1) Pick a sample (x, y) from training data

2) Compute output $\hat{y}$

$$\hat{y} = \boldsymbol{\theta}^T \boldsymbol{x} = \boldsymbol{x}^T \boldsymbol{\theta}$$

3) Compute loss

$$L = (\hat{y} - y)^2$$

4) Compute derivative

$$\nabla_{\boldsymbol{\theta}} \mathrm{L} = 2\boldsymbol{x}(\hat{y} - y)$$

5) Update parameters

$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \mathrm{L}$$

η is learning rate

demo

```python
import numpy as np

# forward
def predict(x, theta):
    return x.dot(theta)

# compute gradient
def gradient(y_hat, y, x):
    dtheta = 2*x*(y_hat-y)

    return dtheta

# update weights
def update_weight(theta, lr, dtheta):
    dtheta_new = theta - lr*dtheta

    return dtheta_new
```

$\hat{y} = wx + b$

$\boldsymbol{x} = \begin{bmatrix} 1 \\ x \end{bmatrix} \quad \boldsymbol{\theta} = \begin{bmatrix} b \\ w \end{bmatrix}$

Feature	Label
area	price
6.7	9.1
4.6	5.9
3.5	4.6
5.5	6.7
x	y

1) Pick a sample (x, y) from training data

2) Compute output $\hat{y}$

$$\hat{y} = \boldsymbol{\theta}^T \boldsymbol{x} = \boldsymbol{x}^T \boldsymbol{\theta}$$

3) Compute loss

$$L = (\hat{y} - y)^2$$

4) Compute derivative

$$\nabla_{\boldsymbol{\theta}} \mathrm{L} = 2\boldsymbol{x}(\hat{y} - y)$$

5) Update parameters

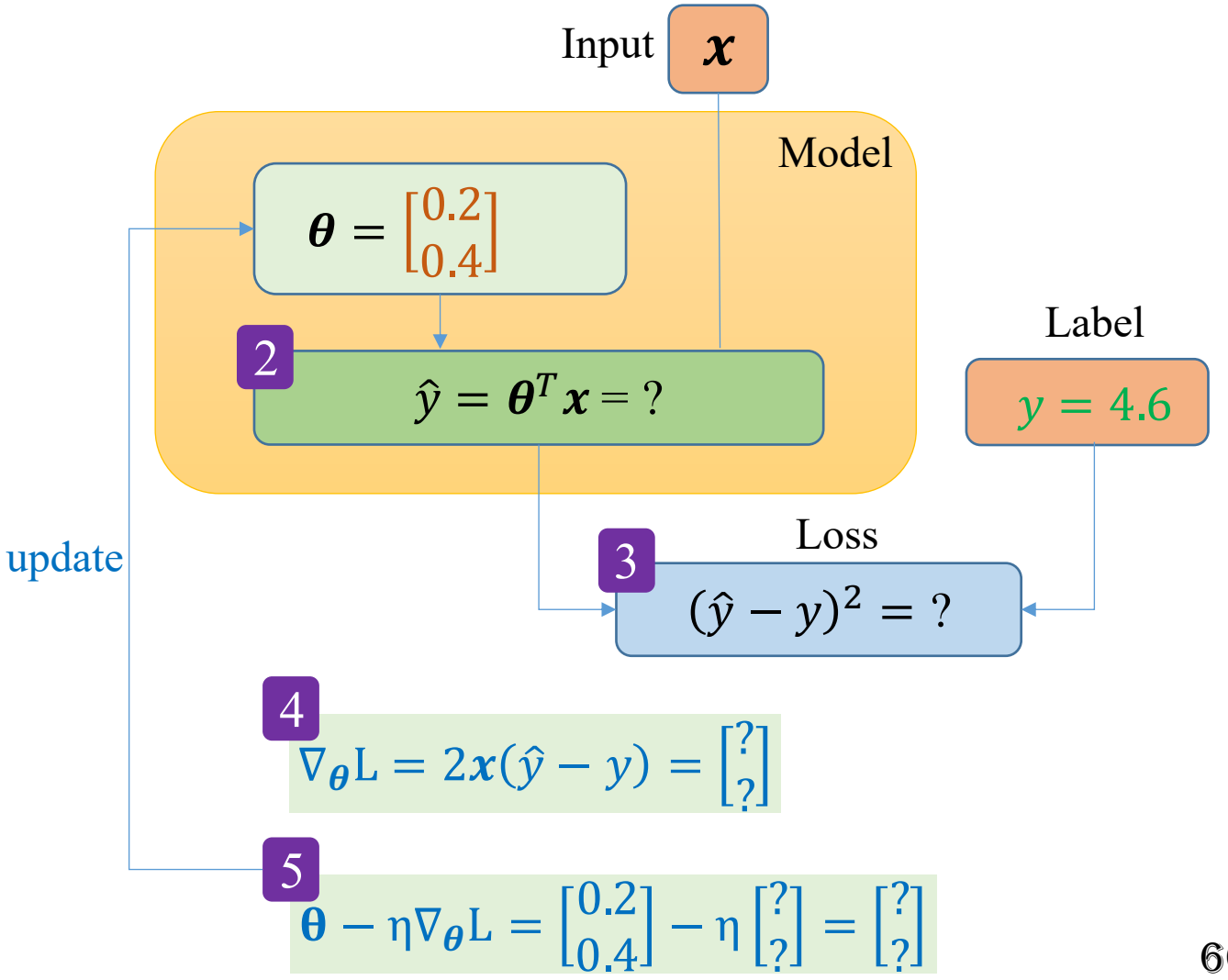
$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \mathrm{L}$$

η is learning rate

1 $\boldsymbol{x} = \begin{bmatrix} 1 \\ x \end{bmatrix} = \begin{bmatrix} 1 \\ 3.5 \end{bmatrix}$

Given $\boldsymbol{\theta} = \begin{bmatrix} b \\ w \end{bmatrix} = \begin{bmatrix} 0.2 \\ 0.4 \end{bmatrix}$

$\eta = 0.01$



```python
data = np.genfromtxt('data.csv', delimiter=',')
N = 4

areas  = data[:, 0].reshape(N, 1)
prices = data[:, 1].reshape(N,)

# vector [1, area]
features = np.hstack([np.ones((N,1)), areas])
```

```python
# forward
def predict(x, theta):
    return x.T.dot(theta)

# compute gradient
def gradient(y_hat, y, x):
    dtheta = 2*x*(y_hat-y)
    return dtheta

# update weights
def update_weight(theta, lr, dtheta):
    dtheta_new = theta - lr*dtheta
    return dtheta_new
```

```python
lr = 0.01
epoch_max = 10

# [b, w]
theta = np.array([0.049, -0.34])

for epoch in range(epoch_max):
    for i in range(N):
        # get a sample
        x = features[i,:]
        y = prices[i]

        # predict y_hat
        y_hat = predict(x, theta)

        # compute loss
        loss = (y_hat-y)*(y_hat-y)

        # compute gradient
        dtheta = gradient(y_hat, y, x)

        # update weights
        theta = update_weight(theta, lr, dtheta)
```

Advertising Problem

$$\hat{y} = w_1 * x_1 + w_2 * x_2 + w_3 * x_3 + b$$

$$x = \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} \qquad \theta = \begin{bmatrix} b \\ w_1 \\ w_2 \\ w_3 \end{bmatrix}$$

Features			Label
TV	Radio	Newspaper	Sales
230.1	37.8	69.2	22.1
44.5	39.3	45.1	10.4
17.2	45.9	69.3	12
151.5	41.3	58.5	16.5
180.8	10.8	58.4	17.9

Advertising data

if TV=55.0, Radio=34.0, and Newspaper=62.0, price=?

1) Pick a sample $(\mathbf{x}, y)$ from training data

2) Compute output $\hat{y}$

$$\hat{y} = \boldsymbol{\theta}^T \boldsymbol{x} = \boldsymbol{x}^T \boldsymbol{\theta}$$

3) Compute loss

$$L = (\hat{y} - y)^2$$

4) Compute derivative

$$\nabla_{\boldsymbol{\theta}} \mathrm{L} = 2\boldsymbol{x}(\hat{y} - y)$$

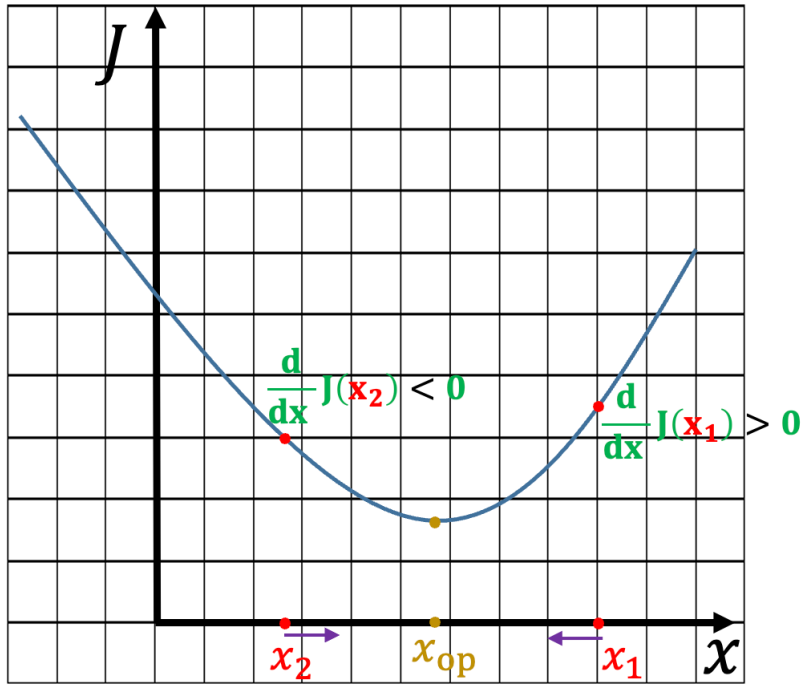
5) Update parameters

$$\boldsymbol{\theta} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \mathrm{L}$$

η is learning rate

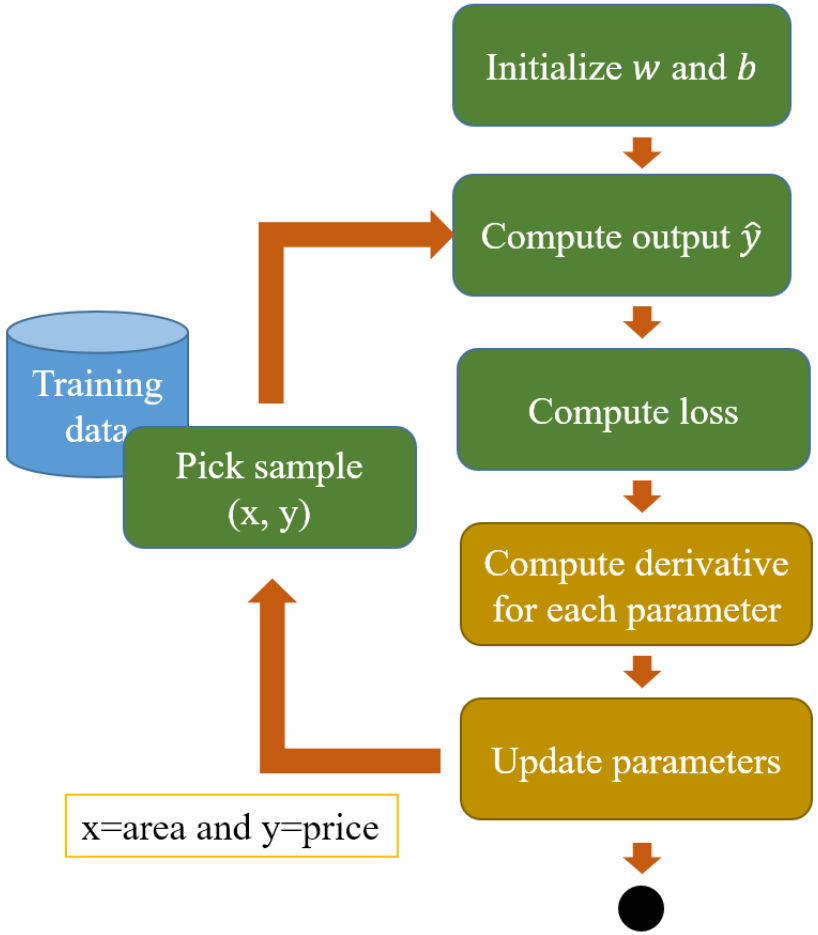
Summary

Gradient descent



$$x_{new} = x_{old} - \eta \frac{d}{dx} J(x_{old})$$

Linear Regression



Vectorization

$$\begin{cases} b = b - \eta \dfrac{\partial L}{\partial b} \\ w = w - \eta \dfrac{\partial L}{\partial w} \end{cases}$$

$$\theta \quad \theta \quad \nabla_\theta L$$

$$\rightarrow \quad \theta = \theta - \eta \nabla_\theta L$$

